Testing And Tdd

. False-positive immunity is an invariant

A test that always returns PASS because its regex never matches, its input path is wrong, its assertion target does not exist, or its comparison is tautological is **worse than no test**. Every new gate MUST be paired with a mutation entry in the project's meta-test harness that:

1. Temporarily breaks the assertion (sed out a line, rename a symbol, etc.).
2. Re-runs the gate.
3. Asserts the gate now reports FAIL.
4. Restores the original.

If the mutation round does not turn PASS → FAIL, the gate is a sham and must be rewritten.

This is the **single most important rule in this Constitution.** Every subsequent §11.4.x clause is downstream of it.

. Commit and push mechanics — single entrypoint, locked

All commit and push work uses the project's official multi-remote commit wrapper (e.g. `scripts/commit_all.sh`) as the single entrypoint. Direct `git commit` / `git push` / `git add` on the main repo is prohibited in normal workflow. Exception: tag creation + tag-ref push is done via explicit `git tag -a` and `git push <remote> <tag>`.

The commit and push wrappers MUST hold an advisory `flock` so two invocations cannot race against each other. Lock files self-clean on process exit via `trap`.

When a contributor sees the "another commit/push wrapper is already running" error, they MUST NOT `rm -f` the lock unless they have just killed the owning process.

§ . . — Test playback cleanup mandate

A test that completes (PASS, FAIL, or SKIP) MUST leave the target in a quiescent state — no orphan playback, no orphan recording, no orphan capture, no orphan background process continuing after the test's last assertion. Tests are transient probes; their side- effects on shared target state MUST NOT outlive the test.

Mandatory protections:

1. Every test that issues a playback or capture command MUST issue the matching cleanup at test end (force-stop the app, stop the recorder, close the file descriptor, terminate the helper).
2. Cleanup is mandatory on EVERY exit path — PASS, FAIL, SKIP, error, trap, signal, parent-killed. Use shell `trap '<cleanup>' EXIT` or `try/finally`.
3. Tests MUST verify the cleanup succeeded via positive evidence (§11.4.5).
4. The orchestrator MUST run a post-test sanity check between tests and FAIL the just-completed test if it left orphan state. This blames the leaker, not the next-test victim.
5. No grace period for "the next test will clean it up" — that is precisely the §11.4 PASS-bluff pattern. Cleanup is the test's responsibility, not the next test's, not the orchestrator's.

§ . . — Full-Automation-Coverage Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!"

Operative rule. For every consuming project under this Constitution, no feature, functionality, flow, use case, edge case, service, or application on any supported platform may be considered **deliverable** until it is covered by automation tests that collectively prove six invariants:

1. **Anti-bluff posture (per §7.1 + §11.4):** every assertion carries captured runtime evidence; metadata-only / configuration-only / grep-based / absence-of-error PASSes are forbidden.
2. **Proof of working capability:** the user-visible behaviour is exercised end-to-end on the target platform topology (per §11.4.3), not in a mock or in-memory facsimile.
3. **Working implementation as expected:** assertions match the product's documented promise (in user manual, README, specs), not an implementation-detail back-door.
4. **No issues, no bugs:** the test suite is the canonical seam for surfacing defects — passing without producing defect signals means defects do not exist or have been previously surfaced, tracked (per §11.4.15 / §11.4.16), and closed.
5. **Fully documented:** the feature has a user-facing doc entry (per §11.4.18 for scripts, and project-level user-manual coverage for everything else); the doc is kept in sync with the tests (per §11.4.12).
6. **Tests-covered (the four-layer floor per §1):** pre-build presence-of-change gate, post-build artifact-shipped gate, runtime / integration / on-device gate, AND meta-test paired mutation that proves the runtime gate catches the break.

Cross-cutting reach. This mandate is **universal** — it applies to every project consuming this Constitution, every feature surface they expose (HTTP endpoints, CLI commands, slash commands, IPC channels, plugins, hooks, MCP servers, agents, providers, integrations, GUI flows, mobile flows, scheduled jobs), and every supported platform (Linux, macOS, Windows, iOS, Android, AuroraOS, HarmonyOS, embedded, containers, headless servers, kiosk, etc.). A project that ships a feature without satisfying all six invariants is **not delivering a stable product**, irrespective of how green its summary line looks.

Coverage audit. Consuming projects MUST publish a coverage ledger (matrix of: feature × platform × invariant-1..6 × status) that is regenerated as part of the release-gate sweep. The ledger itself is documented (per §11.4.18 / §11.4.12) and committed via the §11.4.22 lightweight doc-sync wrapper. Gaps in the ledger (`UNCONFIRMED:` / `PENDING:` / `BLOCKED:` cells) MUST cite a tracked work item per §11.4.15 + §11.4.16; rows that quietly omit a platform are §11.4.25 violations.

Composition. §11.4.25 explicitly stacks on top of §1 (four-layer test-coverage floor), §1.1 (false-positive immunity), §7.1 (positive-evidence-only validation), §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence requirement), §11.4.3 (per-environment-topology dispatch), §11.4.6 (no-guessing), §11.4.15 / §11.4.16 (status + type tracking), §11.4.17 (universal-vs-project classification — this rule is universal), §11.4.18 (script documentation), §11.4.20 (subagent delegation when the work is multi-step), §11.4.22 (lightweight doc-sync). It does NOT supersede them; it forecloses the loophole "the feature works locally for me, ship it".

Classification: universal (per §11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer — a project claiming "Done" without honoring §11.4.25 is making a false claim regardless of how the work-item tracker labels the row.

2026-05-15

§11.4.26 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (User mandate, 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!)."

Operative rule. Two cooperating invariants:

(A) No-fakes-beyond-unit-tests. Mocks, stubs, fakes, placeholders, in-memory facsimile implementations, `TODO`, `FIXME`, "for now", "in production this would", or any empty-implementation pattern are PERMITTED only inside unit-test sources (e.g., `*_test.go` files invoked WITHOUT the integration build tag; `tests/unit/`; equivalent per-language conventions). Every other test type — integration, end-to-end, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA suites — MUST exercise the **real, fully implemented system** against real infrastructure (real databases, real HTTP endpoints, real containers, real downstream services, real captured devices). A non-unit test that imports mocks, in-memory repositories, fabricated provider responses, or placeholder structs is a §11.4.27 violation regardless of how green its summary line looks — severity-equivalent to a §11.4 PASS-bluff. The same prohibition extends to **production code**: no mock import path may be reachable from any code path that runs in production binaries. Pre-build gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` scans the non-unit test trees and production trees for the forbidden patterns; paired mutation plants a fake → gate FAILs.

(B) 100% test-type coverage with every supported type. Every project under this Constitution MUST cover its entire codebase with **every supported test type** the project's domain warrants:

1. **Unit** — fast, isolated, mocks permitted per (A).
2. **Integration** — multi-component, no mocks, real backing services.
3. **End-to-end (E2E)** — full user-flow exercise on target topology.
4. **Full automation** — orchestrated suites exercising every feature × platform combination (§11.4.25 coverage ledger).
5. **Security** — authn/authz boundaries, secret-leak scans (§11.4.10), input-fuzzing, dependency-CVE scanning, threat- model verification.
6. **DDoS** — request-flood resilience at advertised throughput tier, with rate-limit + back-pressure assertions.
7. **Scaling** — horizontal + vertical scale behaviour under linear load growth, including replica add/remove transitions.
8. **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew) verifying graceful degradation paths.
9. **Stress** — sustained load above advertised tier, asserting bounded resource exhaustion + clean recovery.
10. **Performance** — latency / throughput / tail-latency invariants vs SLO baselines.
11. **Benchmarking** — micro + macro benchmark suites with historical p95-drift detection (§11.4.24 build-resource composition).
12. **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface.
13. **UX** — flow-correctness + accessibility + i18n + visual- cue ordering (§11.4.23 composition).
14. **Challenges** — `vasic-digital/Challenges` submodule fully incorporated; per-feature Challenge scripts covering real user use-cases with captured runtime evidence.
15. **HelixQA** — `HelixDevelopment/HelixQA` submodule fully incorporated; ALL written test banks executed; full autonomous QA sessions run as part of release gates.

Per-type 100% coverage means: for every feature × platform cell in the §11.4.25 coverage ledger, the cell carries a verified PASS evidence pointer for **each supported test type the cell warrants** (a CLI-only feature warrants unit + integration + E2E + full-automation + security + chaos + stress + performance

- Challenges + HelixQA; a UI feature additionally warrants UI + UX; a network service additionally warrants DDoS + scaling + benchmarking). Gaps are tracked per §11.4.15 with explicit `UNCONFIRMED:` / `PENDING_FORENSICS:` / `OPERATOR-BLOCKED:` reasons.

(B.1) Seven-canonical-type enumeration (extension, 2026-08-15). The CANONICAL SEVEN-TYPE ENUMERATION per operator mandate 2026-08-15 (§11.4.224(B.1) extension of the same date) is: (1) UNIT, (2) INTEGRATION, (3) E2E, (4) FULL AUTOMATION, (5) SECURITY, (6) PERFORMANCE AND BENCHMARK, (7) ANTI-BLUFF — the MINIMUM breadth every consuming project satisfies. The broader domain-warranted set enumerated in clause (B) above (DDoS, scaling, chaos, stress, UI, UX, Challenges, HelixQA autonomous QA sessions) is ADDED to the seven-type floor per domain, never SUBTRACTED. Applicability of each type is consumer-owned §11.4.35 DATA — a test type genuinely inapplicable is §11.4.3 SKIP-with-reason at the type-breadth ledger, never silently omitted (§11.4.6 no-guessing at the coverage-claim layer). §11.4.27 sets the type BREADTH; §11.4.224 sets the code-coverage FLOOR across that breadth; both bind together, satisfying one is never evidence for the other (per §11.4.224(F) honest boundary). Propagation: existing `CM-COVENANT-114-27-PROPAGATION` stays unchanged (literal `11.4.27` unchanged, stays GREEN); `CM-NO-FAKES-BEYOND-UNIT-TESTS` gate stays unchanged; NEW recommended gate `CM-TEST-TYPE-BREADTH-SEVEN-CANONICAL` — every consuming project's test suite covers all 7 canonical types where applicable, honest SKIP-with-reason where not; a paired §1.1 mutation that drops one type without a SKIP-with-reason → the gate FAILs.

Required submodule incorporation (recursive). Every project consuming this Constitution MUST add the following as Git submodules (or vendored equivalents authorised by the operator), fully recursively per CONST-047 / §11.4.x cascade:

- `Challenges` — `git@github.com:vasic-digital/Challenges.git`
- `HelixQA` — `git@github.com:HelixDevelopment/HelixQA.git`

- Any additional functionality submodules under `vasic-digital/*` or `HelixDevelopment/*` orgs that the consuming project depends on (do not duplicate work the organisations already maintain).

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (§11.4.26 step 7). Pointer drift = §11.4.27 violation.

HelixQA autonomous sessions. "Full HelixQA QA autonomous sessions" means: HelixQA's orchestrator drives the consuming project's running surface end-to-end, executing every test bank the project has registered with it, capturing wire evidence per check (status + body bytes + body-head + duration + screenshots for UI), and producing a session report that is itself committed via the §11.4.22 lightweight doc-sync wrapper. A green autonomous-session report without captured wire evidence is a §11.4 PASS-bluff at the QA-orchestration layer.

Composition. §11.4.27 composes with: §1 (four-layer floor), §7.1 (positive-evidence-only), §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.3 (per-topology dispatch), §11.4.6 (no-guessing), §11.4.10 (credentials-handling — security test category), §11.4.15 / §11.4.16 (status + type tracking), §11.4.17 (universal-vs-project classification), §11.4.20 (subagent delegation for orchestrating the test-type matrix), §11.4.22 (lightweight doc-sync for coverage ledgers and session reports), §11.4.25 (full-automation-coverage — §11.4.27 is its strict expansion into per-type-of-test territory). It does NOT supersede them. CONST-047 (recursive submodule application) governs the cascade reach.

Classification: universal (per §11.4.17). No escape hatch. A project shipping with mocks reachable from non-unit-test paths OR missing required test-type coverage is **not delivering a stable product** — release blocker for every consuming project, severity-equivalent to a §11.4 PASS-bluff at the release-gate layer.

---

§ . . — Per-Feature On-Device End-User Validation Mandate (iter- , - -)

Every user-facing feature in a consuming project MUST have at least one HelixQA scenario that exercises the feature from cold-launch with **positive runtime evidence**:

- A screenshot or screen recording captured at a named checkpoint.
- At least one assertion of type `screenshot`, `accessibility_count`, `accessibility_node`, `ocr_text`, `pixel_histogram`, or `editor_state`.
- Evidence files written to a discoverable output directory so every PASS is cross-verifiable after the fact.

Scenarios MUST be RE-EXECUTED on every release candidate to catch cross-iteration regression. A scenario authored but never re-run on subsequent iterations degrades to a metadata-only gate — a §11.4 PASS-bluff at the regression layer.

Authoring rules (consuming project sets project-specific paths):

1. Scenarios are stored in a canonical bank directory designated by the consuming project (e.g. `<banks-root>/feature-coverage/`).
2. Each scenario YAML MUST include: `name`, `version`, `metadata`, `platforms`, `test_cases`, and at least one step with `evidence_required: true` and a specific `evidence_type`.
3. A **coverage matrix** document MUST exist listing feature × iteration × scenario. Matrix rules: (a) new iteration MUST add ≥ 1 scenario per new user-facing feature; (b) prior scenarios MUST NOT be deleted (mark as `status: retired` if feature is removed); (c) all non-retired scenarios MUST PASS for ship-ready.
4. A **static gate** (challenge script) MUST assert the scenario count ≥ N (where N is the number of user-facing features registered in the coverage matrix) and that each YAML contains a `evidence_type:` assertion.
5. **Portable host tooling**: evidence-validation scripts MUST use POSIX-portable file-size helpers (e.g. `wc -c < file`) instead of OS-specific commands (e.g. GNU `stat -c%s`) to avoid silent 0-byte reports on BSD/macOS hosts. A regression challenge MUST verify the portable helper on the actual host.

iOS / platform-deferred pattern: scenarios written platform-agnostically with the target platform listed in a comment deferral (e.g. `# iOS: deferred — tracker #X`) are valid; add the platform to the `platforms:` list when the automation toolchain becomes available — no structural change to the scenario is needed.

Classification: universal (§11.4.17). Composes with §7.1, §11.4 (anti-bluff), §11.4.25 (full automation coverage), §11.4.27 (no fakes), §11.4.38 (artifact evidence). No escape hatch — a feature without an on-device scenario is NOT covered per §11.4.25 invariant 2. See Constitution §11.4.39 for the full mandate.

§ . . — TDD-Fix-Discipline Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Make sure you do validate and verify every fix! Make sure you first do the fix on live device using ADB if that is possible, once fix is confirmed, make sure that it is fully applied into our codebase ... It is important to note that we shall start by creating test which confirms the issue (fails as expected), and ones the fix is produced it becomes positive - it does pass with success! No bluff policy is mandatory and nothing can be bluffed! Every step of this process MUST BE 100% bluff-free and bluff-proofed!"

Operative rule. Every fix MUST follow the 5-step TDD-fix workflow:

1. **RED — Failing test FIRST.** Before any code change, author a test (or extend an existing one) that exercises the user-visible path of the reported defect and FAILs for that defect specifically. Per §11.4.1 the FAIL MUST be a real product defect (not a script-bug induced FAIL). Per §11.4.2 the FAIL run MUST produce captured evidence (logcat / dumpsys / screenrecord / dmesg / `/sys` snapshot / sink-side probe). The test identifier MUST cite the §-letter or Fix-# under repair. A RED step that "happens to pass because the underlying probe lifecycle isn't wired" is itself a §11.4 PASS-bluff.

2. **LIVE-ADB-PROBE — try the fix on the running device first (when feasible).** For mutable surfaces — `setprop persist.*`, `settings put`, `pm clear`, `am force-stop`, `am start`, `cmd wifi`, `cmd media_session`, ad-hoc `/sys` writes, boot- script edits pushed to `/vendor/bin/`, test-fixture replacement — the operator MUST attempt the fix on the running device via `adb shell` first. The fast adb-loop (~5 min per attempt) replaces the build-flash-test loop (~6 h per attempt) for confirmation of the hypothesis. The live-probe IS NOT the fix; it is the EXPERIMENT that proves the hypothesis is correct before the source-side investment.

   Live-probe is INFEASIBLE (exception, must rebuild) for: kernel changes, AOSP framework (frameworks/base/), hardware HAL changes, vendor/ partition contents, init.rc service definitions, sepolicy, ro.* build properties (immutable post-boot), AOSP-build-time XML / resource overlays, Android.bp/Android.mk changes, signed-system-app contents (LOCAL_CERTIFICATE := platform). Each exception MUST be cited in the commit message as `LIVE_PROBE_INFEASIBLE: <reason>`.

3. **GREEN — apply the fix to source code.** The source patch MUST achieve the same effect on the device as the live probe. Per §11.4.9 the source change is batched with other source-side fixes where appropriate. Build via the project's containerized build (§12.9). Flash via the project's flash script. The bytes on the assembled image MUST land where the post-build gate expects (per §11.4.4(b) four-layer requirement).

4. **VERIFY — re-run the RED test, must now PASS.** Per §11.4.7 demotion-evidence: the PASS MUST come from the SAME conditions that initially FAILed (same device, same firmware NOW carrying the fix, same load profile, same cycle position). Per §11.4.5 the PASS MUST carry positive captured evidence (presence + correctness — RMS amplitude, ffprobe channel count, frame count, OCR text, sink codec state, etc.). Per §11.4.2 video tests MUST cross-check via the recording-analyzer. Per §11.4.42 a reliability check at the iteration count (typically 10 iterations) MUST PASS all iterations — a single intermittent FAIL keeps the item in `In progress` per §11.4.7.

5. **DOCUMENT — update every relevant doc in the SAME commit.** Per §11.4.4(c): `docs/Issues.md` → `docs/Fixed.md` migration with type-aware closure vocabulary per §11.4.33 (Bug → `Fixed`, Feature → `Implemented`, Task → `Completed`); the project's CLAUDE.md Applied Fixes Reference row;

per-version `docs/changelogs/<tag>.md` entry; affected user-facing guides under `docs/guides/`; HelixQA bank entry per §11.4.4(b); `docs/CONTINUATION.md` per §12.10; project memory file if non-obvious.

Composition. §11.4.43 is the workflow that ENACTS the existing covenant. Composes with: §11.4.1 (the RED FAIL is a real defect), §11.4.2 (captured evidence at both RED and VERIFY), §11.4.3 (topology dispatch in the RED test), §11.4.4 (four-layer coverage at GREEN), §11.4.5 (quality analysis at VERIFY), §11.4.6 (no guessing during the LIVE-PROBE hypothesis statement — `UNCONFIRMED:` until probe proves it), §11.4.7 (demotion-evidence at VERIFY — same conditions), §11.4.8 (research citation precedes step 1), §11.4.9 (GREEN is batched), §11.4.40 (full-suite retest is the release-tag-time additional check), §11.4.42 (10-iteration reliability loop IS step 4's inner cycle).

Gate `CM-COVENANT-114-43-PROPAGATION`. Pre-build gate verifies the §11.4.43 anchor is present in every CLAUDE.md / AGENTS.md across the covenant file set. Paired mutation strips the anchor literal from one consumer file → gate FAILs.

Classification: universal (per §11.4.17) — every consuming project's fix workflow.
No escape hatch. No `--skip-red-test`, `--no-live-probe`, `--skip-verify`, `--skip-document` flag exists. "I'll add the test after the fix" is the exact PASS-bluff pattern §11.4 forbids because the test authored after the fix demonstrates only that the test agrees with the fix, not that the test catches the bug.

§ . . — UI-Driven Video Testing Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "We MUST make sure that all video playback testing with 2nd display is performed by fully automatically using the application's UI / UX, with full navigation, choice of video and clicks to real UI buttons to play or switch / stop videos being played! Maybe direct execution of Intents or Broadcasts does not have reported issues! We MUST test ALL WAYS users or Systems may / could play some video contents or streams! For all applications, all video and audio stream types! All supported codecs! EVERYTHING verified with 2nd display on D3 device and every other device with connected 2nd display and Avrus for proper codec(s) used in its Web Interface / Dashboard! Write as much as possible new tests for all supported test types!"

Why the existing video-test set is insufficient. The legacy `test_video_routing.sh` (v33), `test_video_secondary_display.sh`, `test_video_playback_automation.sh` (v1) plus the 53 per_app_video wrappers all dispatch playback via `am start -a android.intent.action .VIEW` or via MediaSession `cmd media_session play` — surfaces that skip the app's own content-selection UI, player-surface allocation, MediaSession lifecycle (start/pause/stop), and back-button cleanup. A defect like §CL routing-race or §CM frozen-frame mid-back-press cannot reproduce via Intent dispatch because Intent dispatch re-launches the app cold each time and `am force-stop` is a clean cleanup path that masks the leak.

Operative rule. Every video-playback test that asserts secondary-display routing MUST traverse the user-equivalent UI path. Mandatory 5-surface coverage per video app:

1. **Surface A — Launch via launcher icon.** `am start -n <launcher-pkg>/<launcher-activity>` then `uiautomator dump` → locate app icon tile → `input tap X Y`. NOT `am start <video-pkg>/<MainActivity>` (skips launcher).
2. **Surface B — Navigate to content list.** Per-app driver script knows the app's home-tab → content-tab swipe / tap sequence. Asserts at least one content tile is visible via `uiautomator dump` content-desc / text match.
3. **Surface C — Select + play.** Tap a specific content tile (driver records the picked tile's content-desc as captured evidence). Wait for player surface visible.

4. **Surface D — In-app control interaction.** Pause via in-app pause button (not `input keyevent KEYCODE_MEDIA_PAUSE`), resume via in-app play button, in-app rapid switch to a second video (covers §CL).
5. **Surface E — Stop via back button.** `input keyevent KEYCODE_BACK` to exit player. Assert §11.4.14 cleanup: VOM `activeDecoder == null`, secondary surface cleared, no orphan MediaSession.

App-coverage matrix: every video-capable app in PRODUCT_PACKAGES MUST have a UI-driven driver script set (Layer 2 per Section 7 design) AND at least one scenario that uses it. Today: 15 apps covered fully in the initial batch (VK Video, MPV, Lampa as fully-implemented; remaining 12 as templated stubs per Section 8). Expansion to all 53 video-app entries is queued as follow-up work under §O Issues.md per-app matrix.

Stream-type matrix: progressive HTTP / HLS / DASH / RTMP / file- local / DRM-protected. Per app, the driver script tags which stream types its picked content exercises. Aggregator emits a coverage matrix per cycle so gaps are visible.

Codec matrix: H.264 / H.265 / VP9 / AV1 / MPEG-2 / MP4V (video) + AC-3 / E-AC-3 / TrueHD / DTS / DTS-HD / MLP / Opus / AAC (audio). Per scenario, the codec assertion is made via `dumpsys media .metrics` (decoder name in event log) AND via Arvus codec-state probe (audio side) per §11.4.13.

Secondary-display verification. On any device with HDMI-A-1 attached (detected via `dumpsys display | grep "Display 2"`):

- Captured-evidence: dual_display_record.sh ON for both displays per §11.4.5
- Assertion: `ffprobe -count_frames` on secondary mp4 reports

> N frames within 3 s of play-action timestamp

- Anti-assertion: `ffprobe -count_frames` on primary mp4 reports ONLY launcher/home pixels during playback window (no video-decoder output)
- VOM cross-check: `dumpsys video_output_manager` shows `activeDecoder != null` during playback and `== null` after stop within 3 s

When secondary absent (D2 default topology): SKIP per §11.4.3 with explicit reason "topology: no secondary display attached". NEVER FAIL on missing topology.

Arvus codec-state mandate. For every test asserting an audio codec (AC-3, E-AC-3, TrueHD, DTS, etc.) — composed with §11.4.13

- §CG:
- During playback window, call `arvus_probe_codec_state` ≥ 3 times at 1 s intervals (transients fade by sample 2-3)
- Call `arvus_screenshot_capture` per §CG to attach visual evidence of the dashboard
- Assert reported codec matches expected via `arvus_assert_codec_format <regex>`
- ARVUS_HOST unreachable (ABK4 not joined / sink off) → SKIP per §11.4.3, never FAIL

Pre-build gates:

- `CM-COVENANT-114-48-PROPAGATION` — asserts anchor literal present in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (42-file scan).
- `CM-AF-UI-DRIVEN-VIDEO-COVERAGE` — asserts the directory `device/rockchip/rk3588/tests/ui_driven/` exists, contains `lib/ui_driver.sh` reference + scenarios/ subdir + per-app driver subdirs for at least the 3 reference apps (vk_video, mpv, lampa), and that `scripts/testing/run_ui_driven_video_suite.sh` is executable.
- Paired mutations: deleting `ui_driver.sh` → CM-AF-UI-DRIVEN- VIDEO-COVERAGE FAILs; stripping the anchor literal → CM-COVENANT-114-48-PROPAGATION FAILs.

No escape hatch. No `--use-intent-shortcut` / `--skip-ui-traverse` / `--legacy-intent-mode` flag exists. The discipline exists because Intent-based dispatch is exactly the class of test that PASSes while users hit real defects (§11.4 forensic anchor). Operators who feel UI-driven tests are "too slow" should land scenario-level parallelism (multi-device fan-out) rather than bypass the discipline.

Composition.

- §11.4.3 — topology dispatch (secondary present vs absent)
- §11.4.5 — captured-evidence quality (every UI tap captured)
- §11.4.13 — Arvus codec-state mandate
- §11.4.14 — playback cleanup verified via UI back-button path
- §11.4.43 — RED-test-first discipline (initial UI tests RED against §CL/§CM)
- §11.4.44 — revision header on every plan/driver doc
- §CG — Arvus dashboard screenshot

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule AND ships any video-capable Android app.

§ . . — Dual-Approach Testing Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Kinopoisk playback of 5.1 audio supported movies MUST BE done via UI / UX full automation and via execution of Intents (or Broadcasts) directly both! We could have shared tests base and specialized part which will run / play the movie with properly chose audio track (5.1). Document everything up to the smallest details! Create as much as needed new tests for all supported test types and Challenges!"

Composition with §11.4.48. §11.4.48 mandated UI-driven traversal for every video routing test, eliminating the Intent-only PASS-bluffs. §11.4.49 REFINES rather than replaces: every feature test ships in BOTH variants — UI-driven AND Intent-driven — over a shared assertion base. Either alone is a §11.4 PASS-bluff for the OPPOSITE half of the stack. UI catches app-side bugs (content selection, player surface allocation, MediaSession lifecycle, in-app overlays, back-button cleanup). Intent catches framework/system-server bugs (Intent extras parsing, MediaCodec.configure hook MIME routing, IVideoOutputManager bind timing, broadcast permission gating, headless cron automation paths).

Operative rule — 5 mandatory elements:

1. **Both variants required.** Every feature test exercising a user-visible behaviour MUST ship both `<feature>_ui.sh` (uiautomator-based, §11.4.48 surfaces A–E) AND `test_<feature>_ intent.sh` (`am start --es` / `am broadcast` -based). Either alone is forbidden.
2. **Shared assertion base.** Codec-state assertions, captured- evidence collection (screen-recording, ffprobe, RMS amplitude analysis), Arvus codec-state probe + dashboard screenshot, and §11.4.14 cleanup MUST be implemented in a single POSIX-sh library (`tests/lib/dual_approach_test_base.sh`) and reused by BOTH variants of every dual-approach test. Code duplication in the shared layer is forbidden.
3. **Specialised driver code.** UI variant uses §11.4.48 per-app driver scripts under `tests/ui_driven/<app>/` . Intent variant uses `am start --es content_id <id> --es audio_track_id <track>` / `am broadcast -a <action>` directly. Each variant's specialised layer is responsible ONLY for "how the playback gets started"; everything downstream of "playback is now happening" routes through the shared base.
4. **Comprehensive documentation per test.** Every dual-approach test set ships with: (a) §11.4.44 revision header in BOTH variant scripts, (b) per-feature contract under `docs/dual_approach/ <feature>.md` describing the captured-evidence pairing and operator-side verification, (c) Issues.md / Fixed.md entry cross-linking both variant paths.
5. **Kinopoisk 5.1 EAC3 is the canonical first implementation.** The shared base + Kinopoisk 5.1 UI variant + Kinopoisk 5.1 Intent variant land in the same batch as this mandate. Subsequent features (Netflix Dolby Atmos, MPV DTS-HD, Lampa+TorrServe HEVC, VLC FLAC) port to the same pattern. The §CN subagent's fix to the underlying decoder pipeline is the FIRST GREEN target of these tests; both variants are RED per §11.4.43 until §CN lands.

Captured-evidence directory contract. Both variants write to mirror-structured directories `qa-results/dual_approach/<F>/<run- ts>/{ui,intent}/` . Identical filenames; orchestrator diffs the two `result.json` files. Status mismatch (UI=PASS but Intent=FAIL or vice-versa) is itself a finding — it pinpoints which half of the stack contains the bug.

Pre-build gates:

- `CM-COVENANT-114-49-PROPAGATION` — anchor literal in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (42-file scan).
- `CM-AF-DUAL-APPROACH-COVERAGE` — `tests/lib/dual_approach_test_base.sh` exists + sources anti_bluff / ui_driver / credentials + exports `dat_init` / `dat_start_capture` / `dat_assert_codec_ state` / `dat_assert_video_frames` / `dat_assert_audio_channels` / `dat_arvus_dashboard_capture` / `dat_cleanup` / `dat_report_ finding`.
- `CM-AF-KINOPOISK-5-1-DUAL-COVERAGE` — both `tests/ui_driven/kinopoisk/kinopoisk_5_1_play_movie_ui.sh` AND `tests/test_kinopoisk_5_1_play_movie_intent.sh` exist + both source the shared base + both reference EAC3 codec + both assert 5.1 / 6-channel.
- Paired mutations (3): strip anchor → propagation gate FAILs; delete UI variant → coverage gate FAILs; delete Intent variant → coverage gate FAILs.

No escape hatch. No `--ui-only` / `--intent-only` / `--skip- dual` flag exists. The discipline exists because either-alone is the PASS-bluff pattern §11.4 specifically prohibits. Operators who feel dual variants are "too slow" should land scenario-level parallelism rather than bypass the discipline.

Composition.

- §11.4.3 — topology dispatch (Arvus reachable, secondary present)
- §11.4.5 — captured-evidence content-quality analysis
- §11.4.6 — UNCONFIRMED tagging for un-verified Kinopoisk element IDs
- §11.4.10 — credentials never echoed, per-device per-service loader
- §11.4.13 — Arvus codec-state mandate (out-of-band evidence)
- §11.4.14 — playback cleanup via EXIT trap in shared base
- §11.4.17 — universal classification
- §11.4.43 — RED-first TDD (both variants RED until §CN fix lands)
- §11.4.44 — revision header on every variant script + design doc
- §11.4.48 — UI-driven traversal (this MANDATE refines, not replaces)
- §CG — Arvus dashboard screenshot via Presenter receiver
- §CB — credentials loader

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule AND ships any Android app whose user-visible behaviour can be triggered through both UI and Intent/Broadcast paths.

§ . . — Deterministic Consistency Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Our anti-bluff / bluff-proofed / proof-driven work MUST satisfy the following: not a single feature, System or application flow, use case, edge case or procedural action(s) MUST NOT partially work, or sometimes work and sometimes not! There is only one truth that MUST BE fulfilled - no matter how many times we repeat all these scenarios with variations in data, System state(s) and speed of execution, results MUST BE consistent and successful without exception! No false positives or bluffing of any kind is allowed! We MUST add as much full automation tests which will use real System and Applications UI and UX with all flows, use cases and edge cases as needed or as much as it is possible!"

Why this anchor exists. §11.4.7 already forbids the vocabulary `intermittent` / `transient` / `flake` in closure narratives, but that's a textual ban — operators could still let a test "PASS once, FAIL once, PASS again" and report only the first PASS. §11.4.50 closes that gap MECHANICALLY by requiring every PASS to come from N identical iterations rather than from a single observation.

Operative rule — 5 mandatory elements:

1. **N-iteration deterministic outcome.** Every test that PASSes MUST have been executed N times (default N=3, N=10 for cycle- validation suites) against the same firmware MD5 + same device
    - same topology and produced IDENTICAL PASS in every iteration. A test whose outcome diverges across iterations is auto-FAIL per this mandate — there is no "first PASSed therefore X was a flake" path. §11.4.7's expanded forbidden-vocabulary list is enforced mechanically here.

2. **Edge-case + flow + use-case coverage.** Every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit and block release at the 99% threshold (ratchet sequence 70 → 85 → 95 → 99 mirrors §11.4.18 the existing project-side docs-coverage ratchet).
3. **Reliability check helper.** Project anti-bluff helper library ships `ab_run_n_times <test_name> <N> <fn> [args...]`:
   - Loops N times, captures exit code + evidence-hash per iter
   - Asserts all N exit codes identical AND all N evidence-hashes identical
   - On any divergence: `ab_fail` with full N-iteration report
   - Per §11.4.7 forbidden vocabulary: NO operator-facing escape path converts a divergent N-iter run into PASS
4. **Feature-coverage-matrix audit.** Project ships `scripts/testing/feature_coverage_audit.sh`:
   - Walks source layers (every public API surface)
   - Walks test layers (test_*.sh references)
   - Emits coverage report at `qa-results/feature_coverage/ <ISO-UTC>/coverage.tsv`
   - Pre-build gate enforces minimum threshold; threshold ratchets up each phase (70 → 85 → 95 → 99)
5. **Composes with every prior anti-bluff anchor.** §11.4.50 does NOT replace §11.4.43 / §11.4.48 / §11.4.49 — it adds the N-iter consistency dimension across all of them. RED-first TDD (§11.4.43) still applies: every new test starts RED and becomes GREEN after the fix lands. UI-driven (§11.4.48) and dual-approach (§11.4.49) still apply: every test ships in both variants AND each variant passes N iterations identically.

Pre-build gates:

- `CM-COVENANT-114-50-PROPAGATION` — anchor literal `§11.4.50` present in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies (≥42-file scan; phase 1 uses 5-canonical-file scan matching §11.4.49 propagation pattern).

- `CM-AF-RELIABILITY-CHECK-WIRED` — `ab_run_n_times` function literal present in `device/rockchip/rk3588/tests/lib/ anti_bluff.sh` AND at least 3 on-device tests source-reference it. The 3-test floor ratchets upward each phase.
- `CM-AF-FEATURE-COVERAGE-MATRIX` — `scripts/testing/ feature_coverage_audit.sh` exists + executable + has run within the last 7 days + most-recent coverage report meets the current threshold.

Paired mutations (3):

- Strip the `§11.4.50` anchor literal from `constitution/CLAUDE.md` via `sed -i 's|11\.4\.50|11.4.MUTATED|g'` → `CM-COVENANT-114-50-PROPAGATION` MUST FAIL.
- Rename `ab_run_n_times` to `ab_run_n_times_DISABLED` in `anti_bluff.sh` via targeted sed → `CM-AF-RELIABILITY-CHECK- WIRED` MUST FAIL.
- Move `scripts/testing/feature_coverage_audit.sh` aside via `mv` → `CM-AF-FEATURE-COVERAGE-MATRIX` MUST FAIL.

No escape hatch. No `--allow-flake`, `--first-pass-suffices`, `--skip-n-iter`, `--skip-coverage-audit` flag exists. The discipline exists because the user mandate is unambiguous: "results MUST BE consistent and successful without exception."

Composition.

- §11.4.1 — FAIL-bluffs forbidden
- §11.4.2 — captured-evidence (every iteration captures its own)
- §11.4.5 — quality analysis (per-iteration content compared)
- §11.4.6 — no guessing (UNCONFIRMED required for any iteration- budget assumption)
- §11.4.7 — forbidden flake/intermittent vocabulary (enforced mechanically by this mandate)
- §11.4.43 — RED-first TDD (every new test starts RED, N-iter applies across the GREEN transition)
- §11.4.46 — validate-before-suite (N-iter applied at single-test scope first)
- §11.4.48 — UI-driven traversal (each UI traversal MUST be N-iter-consistent)

- §11.4.49 — dual-approach (both variants of every dual-approach test MUST pass N iterations identically)

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule.

2026 05 18

---

§ . . — Live-ADB-First Maximization Mandate (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-18):

> "Was it possible to max the fix and test it via ADB or it can be only done by making the fix and reflashing for validation and verification? If such option exists, we should always make the best of it! Making fix on live devices via ADB → Once done commit and push changes, rebuild System and reflash → Validate and verify everything (with the test we write as well). Make sure this idea(s) is (are) part of our root (constitution Submodule) Constitution, AGENTS.MD and CLAUDE.MD if they are not already! EVERY DETAIL IS IMPORTANT!!!! Not a single idea or idea's detail can be ignored, skipped, relativized or bluffed!"

§11.4.51 REFINES §11.4.43 step 2 ("LIVE-ADB-PROBE — try the fix on the running device first") with mechanical enforcement: a per-file- class decision matrix, a classifier helper, and a commit-message footer literal.

Operative rule (5 mandatory elements):

1. **Classify every fix** by rebuild-requirement using the project's per-file-class decision matrix. No guessing (§11.4.6). Every file in the diff cites the matrix row that classified it.
2. **If LIVE_ADB_TESTABLE,** the operator MUST apply the fix to the running device via the appropriate `adb` channel first (`adb push` for scripts, `setprop persist.*` for runtime properties, `mount -o remount,rw /vendor` for boot scripts, `pm install -r` for APKs built locally via gradle). Run the §11.4.43 RED test against the live-probed device, capture positive-evidence PASS,

THEN commit source-side + rebuild + reflash as belt-and-suspenders re-validation. Commit message footer MUST state `LIVE_ADB_VALIDATED: yes` with one-line description of the live-probe sequence.

3. **If REQUIRES_REBUILD,** the operator proceeds directly to source-side fix + rebuild + flash. Commit message footer MUST state `REQUIRES_REBUILD: <reason>` citing the matrix row that classified the file. Categories of REQUIRES_REBUILD: kernel, AOSP framework Java / AIDL, native C++ inside APEX, sepolicy, init.rc, `ro.*` build properties (immutable post-boot), `Android.bp` / `Android.mk` / `BoardConfig.mk`, XML resource overlays, codec XML inside APEX, ramdisk-bundled artifacts.

4. **If mixed batch,** commit footer states `LIVE_ADB_VALIDATED: partial` and enumerates per-file classification. Per §11.4.9 batching is preferred; live-probe the testable files first, then batch with the rebuild-required ones into a single rebuild.

5. **Helper script enforces classification mechanically.** The project provides `classify_fix_rebuild_requirement.sh` which walks `git diff --name-only`, looks up each file against the matrix, and emits the recommended commit-message footer. No operator-facing override flag converts an unclassified path to LIVE_ADB_TESTABLE by default — unmatched paths classify as `REQUIRES_REBUILD: unmatched-path` (safe default per §11.4.6).

Per-file-class decision matrix (canonical):

File class	Rebuild required?	Reason
`*test_*.sh` / `tests/lib/*.sh` (on-device)	NO	adb push to /data/local/tmp/tests/
Host-side test scripts	NO	host script — no device interaction
`atmosphere-*.sh` boot scripts	DEPENDS	adb push to /vendor/bin/ after remount
Markdown docs / Constitution / plans / Issues / Fixed / CONTINUATION	NO	host docs — no firmware impact
Forked-player Kotlin (Presenter, MPV, SmartTube, TorrServe, Lampa, VLC, etc.)	YES via gradle local build + `adb install -r`	APK re-link required but no full firmware rebuild
Framework Java (`frameworks/base/**/*.java`)	YES	system_server requires reboot at minimum
AIDL (`*.aidl`)	YES	stub generation requires recompile
Native C++ (`external/**/*.cpp` , `frameworks/native/**/*.cpp`)	YES	builds into APEX or system library
APEX libraries (`vendor/**/lib/**/*.so` inside APEX path)	YES	squashfs read-only by Mainline integrity
Kernel sources (`kernel-5.10/**`)	YES	requires kernel rebuild + boot.img
sepolicy (`*.te`)	YES	requires policy build
`init.rc`	YES	parsed at boot only
`ro.*` build properties	YES	immutable post-boot
`persist.*` runtime properties	NO	`setprop persist.*` mutable

File class	Rebuild required?	Reason
XML resource overlays	YES	RRO recompile
`media_codecs_*.xml`	YES	APEX-bundled
`display_settings.xml` (user-data)	DEPENDS	adb push possible if SELinux permits
`Android.bp` / `Android.mk` / `device.mk` / `BoardConfig.mk`	YES	rebuilt-into-image arti-facts
Test fixture binary assets	NO	adb push to /data/local/tmp/

Pre-build gates: `CM-COVENANT-114-51-PROPAGATION` (anchor across canonical files) + `CM-AF-CLASSIFY-FIX-HELPER-EXISTS` (helper present + sentinel literals) + `CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER` (advisory WARN that scans recent commits for the footer literal). Three paired meta-test mutations.

Composition.

- §11.4.43 — TDD-fix workflow (§11.4.51 REFINES step 2 with the mechanical classifier).
- §11.4.9 — batch-source-fixes-before-rebuild (compose: live-ADB- test individually, then batch-commit).
- §11.4.6 — no guessing (each classification justified by matrix row; unmatched paths default to rebuild-required, never silent testable).
- §11.4.46 — validate-recent-work-before-post-flash (live-probe IS implicit pre-validation).
- §11.4.48 — UI-driven tests (live-probe CAN use uiautomator over adb shell).
- §11.4.49 — dual-approach (live-probe applies to both UI and Intent variants).
- §11.4.50 — deterministic consistency (live-probe runs N iterations on the live device before commit).

No escape hatch — no `--skip-classify`, `--assume-rebuild`, `--no-footer-required` flag exists. The discipline exists because the user mandate is unambiguous: "EVERY DETAIL IS IMPORTANT!!!! Not a single idea or idea's detail can be ignored, skipped, relativized or bluffed!"

Classification: universal (per §11.4.17). Applies to every project that consumes the constitution submodule — the matrix itself MAY be project-specific (each consumer adapts the file-class list to its own source tree), but the mandate to classify-before-commit + the LIVE_ADB_VALIDATED / REQUIRES_REBUILD footer literals + the classify-helper-script + the three pre-build gates are universal.

2026 05 19

§ . . — Shell-script target-shell-parseability mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-19):

> "any issue we spot must be fixed, bash scripts as well if they are broken!"
> "Make sure that this is mandatory rule!"

Forensic incident — Phase 39.FJ.67 D3 post-flash cycle block: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` invoked on Android via `sh script.sh` used bash-only process substitution `exec > >(tee -a "$file") 2>&1`. Android's `/system/bin/sh` is mksh, which parses the *entire* script BEFORE executing it — so the runtime guard `if [ -n "${BASH_VERSION:-}" ]` could not save the script: mksh rejected `>(...)` at parse time and the test cycle failed to launch on D3. The runtime guard correctly tried to gate the unsafe path, but mksh's compile-time parser made the guard useless. Fixed by wrapping the offending `exec` in `eval` so the parser sees only a string and the parse step defers to runtime where the guard can take effect.

This is the canonical class of script-bug FAIL-bluff: a test cycle PASSes on the developer host (bash) and FAILS at parse time on the target (mksh), producing a FAIL-bluff that masks every product defect the cycle would have caught. §11.4.1 already forbids script-bug FAILs. §11.4.67 mechanically prevents this specific sub-class.

The mandate. Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Specifically:

1. **Closed-set scope.** Every tracked `.sh` file under `device/rockchip/rk3588/tests/`, `scripts/`, and `scripts/testing/` (and equivalent paths in owned submodules) is in scope. Explicitly OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/` (legacy quarantine), and any third-party vendored shell directory the project does not maintain.
2. **Mandatory parseability invariant.** Every in-scope script MUST parse under POSIX `sh -n` on the developer host. POSIX `sh` is the closest portable parser to Android mksh — passing `sh -n` on the host gives high confidence the script parses on mksh.
3. **Bash-only syntax requires deferral.** Process substitution `>(...)` / `<(...)`, `[[ ]]`, `<<<` here-strings, indexed arrays `arr=()`, associative arrays `declare -A`, `${var^^}` / `${var,,}` case-fold expansions, `${var/pattern/replacement}` with bash-only operators, `coproc`, `function name { ... }` (vs POSIX `name() { ... }`), and any other bash-only construct MUST EITHER be wrapped in `eval '...bash-only string...'` so the parser sees only a string OR be guarded by sourcing the file only on bash- detected hosts (`[ -n "${BASH_VERSION:-}" ]` BEFORE the script loads — runtime guards inside a single mksh-parsed script do not work).
4. **Honest shebangs.** A script that contains bash-only constructs without `eval`-deferral MUST declare `#!/bin/bash` (or equivalent) AND its callers MUST invoke it as `bash script.sh` rather than `sh script.sh`. A script with `#!/system/bin/sh` or `#!/bin/sh` shebang MUST be POSIX-clean — no bash-only syntax, no `local` keyword without an mksh fallback, no `read -p`, no `echo -e` without portable equivalent.
5. **`eval`-deferral is the safe pattern.** When a bash-only construct is genuinely needed inside a script that may be parsed by mksh (e.g. `sh script.sh` from a callsite outside our control), wrap the construct in single-quoted `eval`: `eval 'exec > >(tee -a "$f") 2>&1'`. The parser sees a string; the runtime guard around the `eval` decides whether to execute.

6. **Sourced-function `exec` redirection-scoping (extension, research-derived, 2026-07-23).** In bash-family shells, `exec` WITHOUT a command applies EVERY redirection on the line to the CURRENT SHELL, PERMANENTLY. Inside a standalone script that is a self-contained choice; inside a SOURCED / library function — code that runs in the operator's interactive shell or a caller's long-lived shell — it is a host-state mutation. Forensic FACT (genericised, 2026-07-22): a library lock helper opening its lock fd with `exec 9>>"$lock" 2>/dev/null` permanently redirected the INTERACTIVE shell's stderr to /dev/null — one provider launch left the operator's terminal unable to print any error, from any command, for the life of the shell — while the paired unlock helper had used the correct brace form (`{ exec 9>&-; } 2>/dev/null`) all along; the asymmetry was the defect. Marker-probe proof: pre-fix, a marker echoed to stderr after the call never appeared; post-fix it does. The mandate: in ANY sourced/library shell function, an `exec` whose purpose is only to open/close/move file descriptors MUST brace-scope its side redirections — `{ exec 9>>"$file"; } 2>/dev/null` — so error-suppression applies to the fd-operation alone, never to the enclosing shell; a command-less `exec` combining fd-manipulation operands with an additional unbraced redirection in sourced code is a REVIEW DEFECT (§11.4.142) and a greppable pre-build-gate class. Recommended gate `CM-SHELL-EXEC-REDIRECTION-SCOPED` (scan in-scope sourced/library shell sources for command-less `exec` lines that combine fd-open/close operands with additional unbraced redirections → FAIL citing file:line; brace-scoped forms and `exec` WITH a command are exempt — the §11.4.201(1) false-positive guard) + paired §1.1 mutation (unwrap one brace-scoped fd-open back to the bare form → the gate FAILs; gate-code = separate work item). The instrument-side twin of this construct — the SAME `exec ... 2>/dev/null` blinding `bash -x` tracing of the code under test — is catalogued in `docs/guides/shell_instrument_footguns.md` per §11.4.201(12).

Captured-evidence enforcement. Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` walks every `.sh` file under the in-scope directories, runs `sh -n` on each with a 30-second per-file timeout, and FAILs if any script fails to parse. The gate's diagnostic output cites the failing file + the `sh -n` error message so the fix location is unambiguous.

Propagation gate. `CM-COVENANT-114-67-PROPAGATION` verifies the §11.4.67 anchor literal is present across the 44-file consumer fleet (parent CLAUDE.md / AGENTS.md + Containers + 10 owned atmosphere submodules + nested SmartTube SharedModules / MediaServiceCore / MSC-SharedModules + 7 HelixQA submodules). Constitution submodule files (constitution/{Constitution,CLAUDE,AGENTS}.md) are canonical authority — they carry §11.4.67 by definition.

Paired meta-test mutations. (a) Inject a bash-only construct (`exec > >(/bin/cat)`) outside `eval` into a fresh location inside a test script and assert `CM-SCRIPT-TARGET-SHELL-PARSEABLE` FAILs. (b) Strip `11.4.67` literal from one consumer file and assert `CM-COVENANT-114-67-PROPAGATION` FAILs.

Composes with:

- §11.4.1 (FAIL-bluffs equally forbidden — script-bug FAILs that mask product defects are the exact failure mode §11.4.67 mechanically closes)
- §11.4.4 (test-interrupt-on-discovery — a parse FAIL at cycle start IS the discovery event; cycle stops, fix lands, retest)
- §11.4.6 (no-guessing — `sh -n` produces fact, not opinion)
- §11.4.50 (deterministic consistency — a script that parses on one shell and not another fails the N-iteration identical-result requirement)
- §11.4.51 (live-ADB-first — shell-script fixes are LIVE_ADB_TESTABLE: push the fix, re-run on device, capture evidence before commit)

Mandatory protections (no escape hatch):

- **No `--skip-parseability-check` flag.** A script that does not parse under the target shell is broken regardless of intent.
- **No "this script only runs on bash" exception.** If a script may be invoked via `sh script.sh` (and almost all of them can — Android's default `sh` is mksh), it MUST parse under mksh.
- **No "the runtime guard catches it" exception.** Runtime guards inside a mksh-parsed script cannot prevent mksh parse-time rejection. Use `eval`-deferral or shebang-controlled invocation.
- **Fix at source, not in callsites.** A broken script is fixed in the script itself (per §11.4.1), not by individual callsites defensively detecting parse failures.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` + paired mutation.
Propagation gate `CM-COVENANT-114-67-PROPAGATION` + paired mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§ . . — Cross-platform-parity mandate (User mandate, - -)

Forensic anchor — direct user mandate (verbatim, 2026-05-21):

> "Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!"

The mandate. Every consuming project whose `supported-platforms` manifest lists more than one OS MUST, for every feature/test/gate/challenge/mutation that depends on platform-specific primitives, ship a per-OS-equivalent implementation chosen at runtime via `uname -s` (or equivalent platform detection — `[[ $OS == 'Windows_NT' ]]`, `sw_vers -productName`, `/etc/os-release` on Linux distros, etc). A Linux-only gate that ships without macOS / Windows / BSD equivalents (when the kernel of those OSes permits one) is a §11.4 / §107 PASS-bluff at the platform-coverage layer: the test reports GREEN on Linux but tells nothing about whether the feature works for end users on every supported platform.

Three sub-mandates.

(A) Per-OS implementation REQUIRED. Every feature whose Linux implementation uses a cgroup / systemd / `/proc` primitive MUST have a documented per-OS equivalent (POSIX `setrlimit` / `ulimit`, macOS `launchd` plist `HardResourceLimits`, BSD `rctl`, Windows Job Object, etc) chosen via runtime platform detection. The wrapper/library code MUST dispatch to the correct branch with NO operator awareness required — the same operator-path invocation works on every supported platform. Canonical example from `vasic-digital/tmux`: `tmx new -s NAME` dispatches to `systemd-run --user`

`--scope` on Linux and to `scripts/tmx-rlimit-wrapper.sh` (POSIX `setrlimit`) on Darwin — same operator command, OS-correct mechanism per session.

(B) Per-OS tests REQUIRED. Every gate test that exercises platform-dependent behavior MUST have a per-OS branch under `case "$(uname -s)" in`. Each branch MUST exercise the platform's actual native primitive with positive captured evidence per §11.4.2 + §11.4.5 (e.g. read `ulimit -t` inside the session via `send-keys` + `capture-pane` on Darwin to verify `RLIMIT_CPU` applied; read `/sys/fs/cgroup/.../memory.max` content on Linux to verify cgroup `MemoryMax` applied). Platform-conditional `SKIP-with-reason` is acceptable ONLY when the platform genuinely cannot enforce the invariant at all (kernel limit — see (C) below), AND the SKIP message MUST cite the precise kernel/system limitation with a reproducer + a link to the project's honest-gap section in its operator guide (typically `docs/guide/README.md` or equivalent).

(C) Honest kernel-gap citation + adjacent equivalent test REQUIRED. Where a Linux primitive has NO macOS / other-OS equivalent due to a documented kernel/OS limitation (canonical example: XNU does NOT enforce `RLIMIT_AS` / `RLIMIT_DATA` / `RLIMIT_RSS` for unprivileged processes — verified by reproducer in `vasic-digital/tmux` `docs/guide/README.md` §5.6), the test MUST: (1) detect the gap at runtime, (2) `SKIP` with the exact kernel reason + reproducer + link to the project's honest-gap doc, (3) **provide an ADJACENT test that exercises the closest invariant the platform CAN enforce** (e.g. `RLIMIT_CPU` + `SIGXCPU` as the macOS proxy for "process is bounded under load"). The adjacent test MUST itself be anti-bluff per §11.4 with positive captured evidence and a paired §1.1 mutation.

Composes with. §11.4.1 (FAIL-bluffs equally forbidden — a Linux-PASS / Darwin-SKIP-without-honest-reason is a §11.4.81 violation), §11.4.2 (recorded-evidence — each branch's captured evidence is platform-specific), §11.4.3 (per-host-topology dispatch — §11.4.81 strictens §11.4.3 by requiring an equivalent test when the platform CAN enforce, not just SKIP-with-reason for genuine kernel gaps), §11.4.5 (captured evidence quality — platform-specific quality analysis), §11.4.4 (test-interrupt-on-discovery — discovering a Linux-only test without a Darwin equivalent triggers §11.4.81 mid-cycle), §11.4.6 (no-guessing — "platform X probably can't do Y" is a §11.4.6 violation; prove via reproducer or mark `UNCONFIRMED`), §11.4.20 / §11.4.70 (subagent-driven — cross-platform branches are independent enough to

dispatch as parallel subagent work per branch), §11.4.27 (no-fakes-beyond-unit + 100% test-type coverage — platform branches inherit the 100% type-coverage discipline), §11.4.69 (universal sink-side positive-evidence taxonomy — every platform branch must produce a taxonomy-mapped sink-side evidence artefact), §107 (end-user usability — multi-platform projects whose tests cover only one platform deliver no end-user guarantee on the others).

Per-OS implementation equivalence catalogue (canonical examples — not exhaustive).

Linux primitive	Darwin equivalent	BSD equivalent	Windows equivalent
`systemd-run --user --scope` (transient cgroup)	`scripts/<rlimit-wrapper>.sh` using `ulimit -t` (`RLIMIT_CPU`) + `ulimit -u` (`RLIMIT_NPROC`)	`rctl process:0:deny:vmemory-use` (root); `ulimit -v` (unprivileged limited)	Windows Job Object via `CreateJobObjectW` + `SetInformationJobObject`
cgroup `MemoryMax` enforcement	**XNU GAP** — unprivileged `RLIMIT_AS` returns `EINVAL` ; root-only via `launchd` `HardResourceLimits` plist. Adjacent test: `RLIMIT_CPU` + `SIGXCPU` proxy	`rctl process:0:deny:memory-use` (root)	Job Object `JOB_OBJECT_LIMIT_PROCESS_MEMORY`
cgroup `TasksMax`	`RLIMIT_NPROC` via `ulimit -u` (per-user, kernel-enforced)	`rctl user:NN:deny:maxproc`	Job Object `JOB_OBJECT_LIMIT_ACTIVE_PROCESS`

Linux primitive	Darwin equivalent	BSD equivalent	Windows equivalent
`/proc/<pid>/oom_score_adj`	**No equivalent** — Darwin/BSD have no kernel OOM killer (different memory model). SKIP-with-reason; adjacent: rlimit bounds rogue session reach	`/dev/oom`-style not present in stock BSD	None at OS level (process governance via Job Object)
`/sys/fs/cgroup/...` introspection	`ulimit -a` readback inside session + `ps` ancestry	`procstat -r <pid>`	`QueryInformationJobObject`
`kill -KILL` on cgroup process to test scope isolation	`kill -KILL` on tmux server pid directly; verify sibling servers survive via direct socket query	same	`TerminateProcess` + per-job survival check

Operational discipline. When a consuming project adds a new feature whose Linux path lands first, the SAME pull request MUST add the per-OS branches for every other platform in the project's `supported-platforms` manifest (or explicitly classify the platform per (C) — honest gap + adjacent test). Skipping the per-OS branches "for later" is itself a §11.4.81 violation: the gap accrues silently and the project ships Linux-PASS / other-OS-untested code while claiming multi-platform support.

Pre-build gate `CM-CROSS-PLATFORM-PARITY` (when implemented in the consuming project): scans every gate test for `case "$(uname -s)"` blocks, asserts a non-SKIP branch (or honest-gap citation per (C)) exists for each platform in the supported-platforms manifest. Paired §1.1 mutation: strip the Darwin branch from any one test → gate FAILs.

Classification: universal (per §11.4.17). Applies to every project under this Constitution whose supported-platforms manifest lists more than one OS. Single-platform projects are unaffected (no parity gap exists).

Canonical authority: constitution submodule `Constitution.md` §11.4.81.

Non-compliance is a release blocker on multi-platform projects. No escape hatch — no `--linux-only-acceptable`, `--no-platform-parity-required`, `--platform-gap-just-skip` flag exists. A Linux-PASS / other-OS-untested feature in a multi-platform project is a §11.4 / §107 PASS-bluff at the platform-coverage layer.

2026 05 24

---

§ . . — Stress + Chaos Test Mandate (User mandate, - -)

Short tag: `stress-chaos-mandate`.

Forensic anchor (verbatim user mandate, 2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

The mandate. Every fix or improvement landed in a consuming project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. A fix that PASSes its happy-path test but has never been exercised under stress or under fault-injection is a §11.4 / §107 PASS-bluff at the resilience layer: it claims to work but carries no

evidence that real-world adversarial conditions (sustained throughput, parallel contention, partial failure, resource exhaustion, malformed input) leave the fix intact and the user-visible behaviour correct.

Definitions (closed-set, mechanically auditable):

1. **Stress test** — exercises the fix-under-test under sustained or concurrent load above ordinary usage. At MINIMUM one of:
   - **Sustained load** — N ≥ 100 sequential iterations OR ≥ 30 seconds wall-clock continuous load. Per-iteration latency MUST be recorded; percentile distribution (p50/p95/p99) MUST be reported.
   - **Concurrent contention** — N ≥ 10 parallel invocations. All N MUST complete. No deadlock, no resource leak (file-descriptor count, process-table count, RSS), no data race in shared state.
   - **Boundary conditions** — minimum input (empty), maximum input (provider/codec/protocol-limit-bound), boundary input (off-by-one at every size threshold). Each boundary MUST produce a categorised result (success, categorised-error, deterministic-skip) — NEVER an uncaught exception or silent corruption.
2. **Chaos test** — exercises the fix-under-test under failure-injection per the §11.4.69 closed-set evidence taxonomy. At MINIMUM one chaos category appropriate to the fix-class:
   - **Process-death injection** — kill the primary process, dependency process, or upstream service mid-operation. Recovery path MUST be deterministic + categorised (clean error, automatic restart, circuit-breaker open).
   - **Network-fault injection** — drop / delay / reorder packets on the relevant interface mid-call. Categorised error per §11.4.69 (e.g. `category=network` / `category=upstream`).
   - **Input-corruption injection** — corrupt the input file / config / .env mid-test. Test MUST detect + report the corruption — NEVER silently consume corrupted input.
   - **Resource-exhaustion injection** — fill disk to 99 %, allocate memory pressure (OOM-injection), exhaust file descriptors, exhaust sockets. Fix MUST refuse new work cleanly OR degrade gracefully — NEVER crash with uncaught error.

    - **State-corruption injection** — mid-flight database lock loss, mid-flight file-system partial-write error, mid-flight cache invalidation. Recovery path MUST restore consistent state.

Anti-bluff (mandatory):

1. Every stress + chaos test PASS MUST cite a captured-evidence artefact path per §11.4.5 + §11.4.69. Acceptable evidence: per-iteration `latency.json` / `throughput.csv` / `categorised_errors.txt` / `state_delta_snapshot.json` / `process_lifecycle.log` / `recovery_trace.log`. Metadata-only PASS ("all iterations exited 0") without per-iteration latency / failure / state evidence is itself a §11.4 PASS-bluff at the stress-layer.
2. Helper library: every consuming project SHOULD ship a `stress_chaos.sh` (or analogous language-binding library) exposing reusable primitives — `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`, `ab_chaos_drop_network_during`, `ab_chaos_corrupt_file_during`, `ab_chaos_oom_pressure_during`, `ab_chaos_disk_full_during`. Each helper composes with `ab_pass_with_evidence` / `ab_skip_with_reason` per §11.4.69.
3. Chaos-injection cleanup is non-negotiable. A test that corrupts a `.env` MUST restore it in `trap '...' EXIT`. A test that fills the disk MUST `rm` the filler in EXIT. A test that kills a process MUST verify the process is restarted (or explicitly leave it down with operator-visible reason). Cleanup failure = §11.4.14 (test-playback cleanup) violation.

4-layer coverage per §11.4.4(b):

- **Pre-build gate** — stress + chaos test files exist + executable + parseable under sh -n + bash -n per §11.4.67; helper library exists + executable; the fix's pre-build gate cites the stress + chaos test file path.
- **Paired meta-test mutation** — per §1.1, removing the chaos-injection step or stripping the per-iteration evidence-capture → the gate FAILs.
- **On-device test** (if the fix is LIVE_ADB_TESTABLE per §11.4.51) — the stress + chaos test is dispatched against a real device, captured-evidence directory under `qa-results/<run-id>/stress_chaos/`.

- **HelixQA Challenge entry** (if the fix is a user-visible feature per §11.4.4(b) layer 4) — a Challenge bank entry references the same stress + chaos test as the validation route.

Composition with related §11.4 anchors:

- **§11.4 / §107** — stress + chaos test PASS is the strongest end-user-quality signal: it demonstrates the fix survives adversarial conditions the happy-path test cannot reach.
- **§11.4.1** — FAIL-bluffs forbidden. A stress test that crashes with `set -u` because the chaos-injection step left a variable unset is a FAIL-bluff; fix at source.
- **§11.4.5** — captured-evidence content quality applies to stress + chaos evidence too (latency distribution recorded, not just count; error categories enumerated, not just "some errors").
- **§11.4.6** — no guessing. Categorised errors per closed-set. "Probably the network" is forbidden — capture the categorisation.
- **§11.4.43** — TDD RED-first. The stress + chaos suite SHOULD start RED: write the test, observe the failure under load/chaos, land the fix, observe the GREEN.
- **§11.4.50** — deterministic consistency. The stress test's N iterations MUST produce identical exit codes AND identical evidence-hashes (volatile prefixes stripped) per §11.4.50.
- **§11.4.52** — autonomous validation. Stress + chaos tests MUST run end-to-end without operator presence.
- **§11.4.69** — universal sink-side positive-evidence taxonomy. Every stress + chaos PASS uses `ab_pass_with_evidence` with a real captured-evidence file path.
- **§11.4.83** — `docs/qa/` end-user transcript discipline composes with chaos-test evidence (chaos-test recovery transcripts ARE end-user-channel proofs).

Classification: universal (per §11.4.17). Every consuming project carries this rule. Stress + chaos discipline is language-/platform-/build-system-agnostic.

Canonical authority: constitution submodule `Constitution.md` §11.4.85.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-stress`, `--no-chaos`, `--happy-path-suffices`, `--stress-test-later` flag exists. The discipline exists because the User mandate is unambiguous: "Everything must produce rock solid proofs and follow fully no-bluff policy!" — and a fix never tested under stress + chaos is not a rock-solid proof.

2026 05 28

§ . . — Full-Automation Anti-Bluff Mandate — Live tests MUST be re-runnable end-to-end without manual intervention (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-05-28):

> "Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! This is important to be done like this! It is critical! Continue all work and make this happen! We need such full automation testing so the whole System MUST BE fully valoidated and verified before it is integrated to our main projects! Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected!"

§11.4.98 composes with §11.4 / §11.4.2 / §11.4.5 / §11.4.50 / §11.4.85 / §11.4.87 / §11.4.89 / §11.4.94 — closes the **manual-intervention gap** that they did not explicitly forbid. A live/integration/e2e/Challenge test which requires a human action during execution (typing a chat message, clicking a UI, hand-triggering a webhook, manually attaching a file, anything beyond the test starting up and reporting PASS/FAIL on its own) is **by definition a §11.4 PASS-bluff at the automation layer**, regardless of how thorough the manual run is. The reason: such a test cannot run

continuously in CI, cannot validate regressions between manual runs, and the human dependency masks any drift between code changes and what the test exercises.

(A) Binding rule. Every test that this Constitution governs — unit / integration / e2e / Challenge / stress / chaos / live — MUST be fully self-driving end-to-end. The test process must accept inputs from configuration alone, exercise the load-bearing code path autonomously, and report PASS/FAIL/SKIP-with-reason without any further human action.

(B) Single permissible exception — one-time credential bootstrap. Configuration performed OUTSIDE test execution is acceptable: populating `.env` from a vault, exporting shell env vars in `~/.bashrc`, OAuth approval at first install, MTProto session activation at first run. This is configuration, not test driving. Once credentials are present, the test MUST run end-to-end without any further human action. The configuration step is itself a §11.4.10 credential-handling concern, not a §11.4.98 automation concern.

(C) Concrete requirements for live messenger / channel / agent tests (the surface that exposed this mandate):

1. **No "operator MUST type a message" prompts in test scripts.** Tests requiring inbound messages MUST drive those messages programmatically — via a separate user account (MTProto for Telegram, real-user-token API for Slack, IMAP-test-account for email, etc.), via a webhook fixture, or via an in-process loopback. Never via human keystrokes during test execution.
2. **No hard-coded session UUIDs that collide with the active dev session.** Test runs MUST use a dedicated test-only session that the production / dev session is NOT simultaneously using. (Lesson from Herald 2026-05-28: `claude --resume <UUID>` invoked against the same session ID the dev session is using returns exit -1 with no output — a silent collision that fails for non-obvious reasons.)
3. **No 60-second human-response windows.** These create flake/timing races that are themselves §11.4.50 determinism violations: a single test invocation's PASS/FAIL depends on a human reaction-time variable, not on the code under test.

4. **Re-runnability proof.** Every live test MUST PASS at least 3 consecutive automated invocations (`-count=3` for Go, `repeat 3` equivalent for other runtimes) with no state cleanup between runs. Persistent side effects (chats, files, DB rows, attached objects, queued events) MUST be cleaned by the test itself in a `defer` / `teardown` block.
5. **§11.4.98 obsolescence audit.** Every existing test that ships under this Constitution MUST be classified as either "self-driving" (COMPLIANT) or "manual-dependency" (NON-COMPLIANT — must be rewritten or marked §11.4.90 Obsolete). The audit is a release-gate item — a release that ships a NON-COMPLIANT test still pretending to PASS is blocked.
6. **No false-positive PASS.** A test PASS that does NOT exercise the load-bearing code path — e.g. a "live" test that skips silently because env vars are missing yet reports PASS, or a "live" test where the captured evidence is from a prior cached run — is itself a §11.4 PASS-bluff at the gating layer. SKIP-with-reason is the §11.4.3 correct posture; SKIP-reported-as-PASS or stale-evidence-reported-as-fresh is forbidden.

(D) Composition. §11.4.85 (stress + chaos) + §11.4.89 (background-test execution) + §11.4.87 (endless-loop autonomous work) + §11.4.94 (zero-idle parallel-by-default) + §11.4.98 (full-automation) together make this Constitution's testing surface a continuously-validated, fully-automated, non-flake, anti-bluff regime. Each closes a different gap; remove any one and the whole property collapses. §11.4.98 specifically closes the manual-intervention gap: §11.4 + §11.4.85 + §11.4.89 + §11.4.87 + §11.4.94 do not forbid a test from requiring human action, only from skipping the load-bearing path. §11.4.98 makes the human dependency itself a §11.4 violation.

(E) Inheritance per §11.4.35. §11.4.98 propagates to every consuming repository via the existing inheritance gate: each project's CLAUDE.md / AGENTS.md / QWEN.md MUST carry a short-form restatement citing the literal anchor `11.4.98`. The pre-build gate `CM-COVENANT-114-98-PROPAGATION` (when implemented) enforces this literal anchor presence across the canonical fleet. Paired §1.1 meta-test mutations strip the load-bearing literal → gates FAIL.

(F) Enforcement. A commit that adds or modifies a test that requires manual human action during execution is blocked at release-gate. A test classified as "manual-dependency" in the §11.4.98 audit that has not been rewritten within 30

days of classification graduates to §11.4.90 Obsolete and is removed from the active test suite (not deleted — preserved with `Obsolete-Details:` per §11.4.90, citing §11.4.98 as the obsolescence reason).

Canonical authority: this Constitution.md §11.4.98 in the HelixConstitution submodule. All consuming projects (Herald, a consuming project, future) restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker. No `--manual-test-OK`, `--skip-114-98-audit`, `--bluff-tolerance-temporary` flag exists. The 2026-05-28 user mandate is unambiguous: "No bluff is allowed!"

1 1 8

2026 06 03

§ . . — Last-known-good-tag regression isolation mandate (. . -dev remediation, - -)

Short tag: `last-known-good-tag-isolation`.

Forensic anchor (genericised, 2026-06-03). A user reported that a previously-working feature (multichannel HDMI audio) was broken on the current build. The operator's lead — "the feature WORKS and passes all tests at the last release tag" — converted an open-ended root-cause hunt into a targeted regression repair: diffing the broken working tree against the last-known-good tag's commit isolated EVERY reported audio defect (multichannel downmix + output-device label mismapping) to ONE post-tag burndown batch (`AudioDeviceInventory.connectDualHdmiPorts_l` dual-HDMI-connect) in minutes, not hours. The tagged-good version is the oracle: behaviour that works there and is broken now is, by definition, a regression introduced in the diff between the two.

The mandate. When a previously-working feature, behaviour, or capability is observed broken, the FIRST diagnostic action MUST be to identify the last release tag (or last commit / build / deployment) at which that feature was KNOWN-GOOD, and diff/bisect the broken state against it — BEFORE any open-ended root-cause investigation, BEFORE any speculative fix. The known-good revision is the regression oracle: (1) it bounds the search space to the commits between known-good and now (a `git diff <good-tag>..HEAD --stat` of the feature's files is the captured evidence); (2) it tells you the fix is a regression REPAIR (restore/

forward-fix the broken sub-part), not a from-scratch design problem; (3) each suspect file is compared against its known-good version as the behavioural oracle. When the operator volunteers a known-good tag, that lead is load-bearing and MUST be acted on first. Where the bisect surface is large, `git bisect` against the feature's autonomous RED test (per §11.4.115) mechanises the search.

Forward-fix vs wholesale-revert. Once the regressing change is isolated, the default is a SURGICAL forward-fix — keep the new features the post-good-tag work introduced, revert ONLY the broken sub-part where the known-good version was correct — UNLESS the operator prefers wholesale revert. A wholesale revert of a multi-feature batch to reach the known-good state is a last resort that loses the batch's other (working) features; prefer isolating and repairing the single regressing seam. Every forward-fix MUST prove (via the §11.4.115 RED→GREEN test) that the feature now matches the known-good behaviour.

Honest boundary (§11.4.6). "It worked before" is a HYPOTHESIS until the known-good tag is identified AND the feature is confirmed working there (re-run the feature's test against the known-good build, or cite the captured-evidence from when that tag shipped). A feature that NEVER worked is not a regression and this anchor's oracle does not apply — investigate as new work. "Probably regressed in the last batch" without the diff is a §11.4.6 guess.

Classification: universal (§11.4.17) — diff-against-last-known-good is a platform-neutral regression-isolation discipline reusable by ANY versioned project with release tags / commit history; the consuming project supplies its tag-naming convention and per-feature known-good oracle per §11.4.35.

Composes with §11.4.4 (test-interrupt-on-discovery — the isolation runs as the first step of the systematic-debugging Phase 1 the STOP triggers), §11.4.6 (no-guessing — the diff is the FACT-grade evidence that replaces "probably the last batch"), §11.4.7 (demotion-evidence — confirming known-good requires positive evidence at that tag), §11.4.40 (full-suite retest authority — the known-good tag is the baseline a release-gate retest restores toward), §11.4.43 (TDD-fix — the §11.4.115 RED test reproduces the regression the diff isolated), §11.4.102 (systematic-debugging — last-known-good isolation IS the Phase 1 root-cause technique for regressions specifically), §11.4.108 (four-layer fix-verification — the forward-fix's runtime signature is verified against the known-good behaviour on a clean target).

Propagation. Propagation gate `CM-COVENANT-114-114-PROPAGATION` enforces the literal anchor `11.4.114` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-REGRESSION-ISOLATED-AGAINST-KNOWN-GOOD` (every tracker entry classified as a regression cites the last-known-good tag + the diff range it was isolated to); paired §1.1 mutation strips the known-good citation from a regression entry → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.114 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-known-good-diff`, `--root-cause-from-scratch`, `--assume-regression-source`, `--wholesale-revert-without-isolation` flag exists.

1 1 8

2026 06 03

§ . . — RED-baseline-on-the-broken-artifact + polarity-switch mandate (. . -dev remediation, - -)

Short tag: `red-baseline-polarity-switch`.

Forensic anchor (genericised, 2026-06-03). Per §11.4.43 a fix opens with a RED test. The 1.1.8-dev remediation sharpened HOW: the autonomous RED test was authored to REPRODUCE each live defect on the CURRENT pre-fix build — proving the defect is real, live, and captured (video flips to primary on reconfigure 3/3; audio 5.1 → Arvus PCM2/0 ×3; subtitle absent on secondary) — with a single environment flag (`RED_MODE=1` default) that, set to `0` post-fix, flips the SAME test to GREEN-guard polarity. One test source, two roles: the reproduction-of-the-defect (RED) and the permanent regression-guard (GREEN). No separate happy-path test is authored; the test that catches the bug IS the test that guards against its return.

The mandate. Every §11.4.43 RED test MUST be authored to reproduce the defect on the CURRENT, pre-fix artifact (the actual broken build / deployment), capturing positive evidence per §11.4.5 / §11.4.69 / §11.4.107 that the defect is genuinely present — never a synthetic / hypothetical failure the fix is then written to agree with. The SAME test source MUST carry a single polarity switch (an env flag / parameter, canonical `RED_MODE`, default `1` = reproduce-and-assert-defect-

present) that, flipped to `0` post-fix, converts the test into the GREEN regression-guard asserting the defect is ABSENT. The RED run (against the broken artifact) and the GREEN run (against the fixed artifact, per §11.4.108 on a clean target) MUST both be captured as evidence: RED-then-GREEN is the proof the test catches the bug AND the fix closes it. A separate happy-path test that never failed on the broken artifact is forbidden as the primary guard — it demonstrates only that the test agrees with the fix (the §11.4.43 PASS-bluff), not that it catches the defect.

Why one test, two roles. A test authored only as a post-fix happy-path can pass on a build where the feature is silently broken (it asserts the fix's intended state, not the defect's absence). A RED-first test that was OBSERVED failing on the broken artifact, then flipped to GREEN-guard, is provably sensitive to the exact defect — its RED capture is the falsification record. The polarity switch keeps the two roles in ONE source so they cannot drift apart (a separate happy-path test can be edited to stop testing the real condition; the RED-evidence binds the GREEN guard to the actual defect).

Honest boundary (§11.4.6). If the defect cannot be reproduced on the current artifact (the RED run does not fail), that is a finding, not a license to write a GREEN-only test: either the defect is not present on this build (close per §11.4.7 with the negative evidence) or the test does not exercise the real condition (fix the test). A RED test that "passes" on the known-broken artifact is itself a §11.4.1 FAIL-bluff inverse — it proves the test is blind, not that the feature works.

Classification: universal (§11.4.17) — RED-baseline-on-the-broken-artifact + a single-source polarity switch is a platform-neutral test-construction discipline reusable by ANY project doing TDD-fix; the consuming project supplies its polarity-flag mechanism (env var, build flag, parameter) and per-defect reproduction driver per §11.4.35. Strict refinement of §11.4.43 step RED.

Composes with §11.4.1 (FAIL-bluffs — a RED test that passes on the broken artifact is blind), §11.4.2 / §11.4.5 / §11.4.69 / §11.4.107 (the RED reproduction captures positive defect-present evidence), §11.4.4 (the RED test is the test-interrupt reproduction), §11.4.7 (RED-not-reproducible is a demotion that needs negative evidence), §11.4.43 (refines its RED step — RED is on the real broken artifact, GREEN is the same test flipped), §11.4.50 (deterministic consistency —

both RED and GREEN runs are N-iteration stable), §11.4.108 (the GREEN run is the runtime-signature verification on a clean target), §11.4.114 (the RED test mechanises the §11.4.114 bisect against the known-good oracle).

Propagation. Propagation gate `CM-COVENANT-114-115-PROPAGATION` enforces the literal anchor `11.4.115` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-RED-POLARITY-SWITCH-PRESENT` (every regression-fix RED test carries a single polarity switch + a captured RED-on-broken-artifact run before the fix); paired §1.1 mutation removes the polarity switch (splitting RED and GREEN into a happy-path-only guard) → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

(F) Machine-written verdict pairs + guard-viability (extension, research-derived, 2026-07-17). Forensic FACT (genericised, from a consuming project's fix-lifecycle forensics, 2026-07-17): 69 commits in one two-month window moved fix claims/statuses at "GREEN-PENDING-BUILD" — the RED→GREEN discipline existed as prose, but the verdicts were hand-declared, so nothing distinguished a polarity flip that HAPPENED on the target from one that was ASSERTED; the project's own history then selected the discriminator — every fix whose done-claim was preceded by an on-device polarity flip that held recorded ZERO reopens, and every fix confirmed at source-green recurred. Therefore (ALL hold): (1) **RED and GREEN are machine-written verdict artifacts, never prose** — the guard HARNESS (not a human, §11.4.205(4)) writes a per-run verdict file carrying at minimum the item id, guard identity, polarity, exit code, the target's artifact/build fingerprint READ FROM THE TARGET AT RUN TIME, iteration count, and the evidence-file list (path + hash + feature-class); a hand-authored verdict is detectable because the fingerprint/target identity must re-verify against the artifacts it cites. (2) **The pair must cross-validate** — GREEN is acceptable only with a prior RED whose exit ≠ 0 on the PRE-fix artifact; GREEN's fingerprint ≠ RED's fingerprint (an identical fingerprint proves the fix was never deployed); GREEN runs on a clean target per §11.4.108/§11.4.139 with iterations ≥ 3 per §11.4.50. (3) **Guard viability** — a guard that has NEVER been observed FAILing on the genuinely-broken artifact is unvalidated instrumentation (the chaos-engineering validate-the-detector principle): it mints no verdicts; its RED run on the real defect IS its §1.1 mutation evidence (the real defect is the mutant, and the guard provably killed it); corollary: a guard structurally unable to FAIL — or unable to PASS (e.g. a pass-floor above the input population's size) — produces no information in either direction and

satisfies nothing. (4) **Mutation = revert + anti-tautology** — the canonical §1.1 mutation for a landed fix is the fix-commit's revert (its semantic inverse, restoring ALL enable-paths of the defect per §11.4.194); a mutation whose diff only deletes/renames the literal strings the gate greps for is a tautology ("grep for X fails after you delete X") and is REFUSED — measured failure: a sed-string mutation "validated" a source-grep gate for a rendered-pixel defect while the defect's second enable-path stayed invisible to both. (5) **Oracle-class** — the verdict's evidence files MUST match the defect's feature-class evidence shape (pixel-class defect ⇒ pixel/capture artifacts; audio ⇒ sink-side/captured-audio; runtime ⇒ runtime observables per §11.4.69); a source-grep transcript can never satisfy a user-visible class — the wrong-layer oracle dies by construction, not by review. Recommended gate `CM-GUARD-VERDICT-MACHINE-WRITTEN` (every done-claiming item's verdict pair is harness-written, fingerprint-cross-validated, class-matched) + paired §1.1 mutation (forge a verdict whose fingerprint does not re-verify, or pair a gate with a string-deletion mutation → the gate FAILs; gate-code = separate work item). Composes §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107(10) / §11.4.108 / §11.4.135 / §11.4.139 / §11.4.146 / §11.4.194 / §11.4.205.

(G) Defect-identity binding for CONSTRUCTED reproductions — a fix validated only against an investigator-built mechanism is HARDENING, never the fix for the reported defect (extension, research-derived, 2026-07-23). Forensic FACT (genericised, 2026-07-22): an exit-hang fix (a bounded lock wait + a corrected file-descriptor open) was built and nearly shipped with a confident RED→GREEN pair — but the RED reproduced a mechanism the INVESTIGATOR had constructed (a hand-made process holding the lock), not the operator's reported defect; later systematic debugging (§11.4.102), driving the operator's sequence shape per §11.4.199 with per-step trace timing on a real PTY, REFUTED every mechanism the fix addressed (the lock/unlock pair provably completes on the real path — 6/6 clean interactive exits), and the change was honestly RE-LABELLED in-source as "defensive hardening ... NOT a fix for the reported exit-hang". The gap clauses (A)–(F) leave open: the constructed RED genuinely FAILED on the real pre-fix artifact — this anchor's letter was satisfied — while the MECHANISM it reproduced was the investigator's invention, so the pair proved only that the fix closes the invented mechanism, not the reported defect. Therefore (ALL hold): **(1)** a RED test's PRECONDITION MUST be TRACEABLE to the reported defect's real evidence — the reporter's reproduction sequence (§11.4.199), a captured failure log/trace, or a probe showing the precondition actually occurs in

the reported environment — and that traceability is recorded WITH the RED verdict (the clause-(F) fields plus a precondition-provenance field: `observed` vs `constructed`). **(2)** When NO working reproduction of the reported defect exists and the RED's precondition is CONSTRUCTED, the resulting fix mints AT MOST "defensive hardening" — labelled as such in the commit AND the tracker — and NEVER closes the reported defect's item; the item stays OPEN (§11.4.197) until a RED with an `observed` precondition lands, or the item is closed with §11.4.7-grade non-reproduction evidence or a §11.4.90/§11.4.112 evidence-backed reason. **(3)** When systematic debugging REFUTES every mechanism a pending fix addresses, the fix's done-claim for the reported defect is REVOKED on the spot — the change may still land re-labelled as hardening (independently crossing §11.4.142/§11.4.194 review), but claiming it fixes the reported defect after its mechanisms were refuted is a §11.4/§11.4.1 bluff at the investigation layer even though its RED→GREEN pair is real. STRENGTHENS clause (A)'s "never a synthetic failure" from artifact-synthetic to MECHANISM-synthetic, and §11.4.199 for the no-working-repro case its clause (1) cannot reach (there is no existing sequence to deviate FROM). Recommended gate `CM-RED-PRECONDITION-DEFECT-TRACEABLE` (every defect-CLOSING claim's RED verdict carries precondition-provenance `observed` + its evidence path; `constructed` provenance on a defect-closing claim → FAIL, while the SAME provenance on an item honestly labelled hardening PASSES — the §11.4.201(1) false-positive guard) + paired §1.1 mutation (flip a `constructed`-provenance item from hardening to a defect-closing status → the gate FAILs; gate-code = separate work item). Composes §11.4.6 / §11.4.7 / §11.4.102 / §11.4.142 / §11.4.146 / §11.4.197 / §11.4.199 / §11.4.201.

(H) Evidence-record full field set + stream-digest-before-truncation + recorder reconciliation (extension, spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). Clause (F) already mandates a machine-written verdict carrying, at minimum, the item id, guard identity, polarity, exit code, the target's artifact/build fingerprint, iteration count, and the evidence-file list — but it does not fix a COMMAND-EXECUTION field schema precise enough to RECONSTRUCT exactly what ran, where, and for how long, nor does it address a captured stream that is truncated or redacted for storage, nor whether a CITED command genuinely has a matching recorder entry. This clause closes those three gaps. **(1) Full command-execution field set** — every evidence-record entry representing a command execution MUST carry, at minimum: a START TIMESTAMP (the moment of invocation, not merely "a timestamp"); the WORKING DIRECTORY at invocation;

the ARGUMENT LIST as a genuine list (never a shell-joined single string — a joined string cannot distinguish an argument containing a space from two separate arguments, the exact class of ambiguity §11.4.201(7)(c) already names as a path-is-part-of-the-instrument footgun); EXIT STATUS; and DURATION (wall-clock elapsed). An entry missing, or carrying an unparseable value for, any of these five fields is REFUSED at the write seam — never silently accepted as a partial entry. **(2) Stream digest computed BEFORE truncation** — when a captured stdout/stderr stream is large enough that the evidence store truncates or redacts it for storage, the entry MUST record a content digest computed over the FULL stream BEFORE any truncation, the full stream's byte count, and a reference to where the full stream is retained OUTSIDE version control (per §11.4.11 file-layout discipline — raw logs stay untracked by default) — so a later dispute can retrieve the full stream and verify it against the recorded digest. Truncation and redaction are themselves RECORDED FACTS on the entry (a flag + the truncation point), never silent. **(3) Recorder reconciliation** — any evidence-bearing claim that cites a specific command's output (a PASS citing a command, a closure citing a captured log) MUST have a MATCHING entry in the evidence recorder for the EXACT command cited; a citation with no matching recorder entry is REFUSED at the SAME status-write seam §11.4.146(D3) already enforces for terminal statuses — the reconciliation check runs AUTOMATICALLY at that seam, never as a manual audit step. **Honest boundary (§11.4.6).** This clause specifies the field SCHEMA and the truncation/reconciliation disciplines; it does not alter clause (F)'s cross-validation rules (fingerprint match, iteration count, evidence-class match), which remain in force unmodified. Recommended mechanism gates `CM-EVIDENCE-RECORD-FIELD-SET-COMPLETE` (every command-execution entry carries start-timestamp/cwd/argv-as-list/exit-status/duration; a missing or unparseable field refuses the write) + `CM-EVIDENCE-STREAM-DIGEST-PRE-TRUNCATION` (a truncated/redacted stream entry carries a full-stream digest + byte count + an outside-VCS reference, all computed BEFORE truncation) + `CM-EVIDENCE-RECORDER-RECONCILED` (every cited command has a matching recorder entry; an unreconciled citation is refused at the consuming seam) + paired §1.1 mutations (strip the argument-list field and store a joined string instead → the field-set gate MUST FAIL; digest the stream AFTER truncation instead of before → the digest gate MUST FAIL; cite a command with no matching recorder entry → the reconciliation gate MUST FAIL; golden-FALSE per §11.4.201(1): a fully-fielded, pre-truncation-digested, reconciled entry → none of the three gates fire). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: this Constitution.md §11.4.115 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--green-only-test`, `--skip-red-reproduction`, `--separate-happy-path-suffices`, `--synthetic-red-OK` flag exists.

2026-06-03

§11.4.116 — Real-time conductor ↔ autonomous-test-framework sync channel mandate (dev-time remediation, not runtime)

Short tag: `conductor-test-sync-channel`.

Forensic anchor (genericised, 2026-06-03). An autonomous test / QA framework (HelixQA) was wired into the conductor's loop. For the conductor (the orchestrating agent / operator) to stay in real-time sync with a long-running autonomous session — knowing which phase / challenge is executing, what evidence was captured, which LLM/vision call ran, and the per-item PASS/FAIL/SKIP/OPERATOR-BLOCKED verdict the moment it lands — the framework exposed a structured append-only JSONL event stream (`conduit.events.jsonl`) plus an atomically-rewritten status snapshot (`conduit.status.json`) that the conductor tails live. Without it, the conductor either polls blindly (idle waiting, §11.4.94 violation) or learns the outcome only at session end (no mid-session course-correction, no §11.4.4 test-interrupt-on-discovery).

The mandate. Any autonomous, long-running test / QA / validation framework that an external orchestrator (conductor agent or operator) depends on for real-time decisions MUST expose a real-time sync channel with two parts: (1) a **structured append-only event stream** (JSONL or equivalent, one event per line, never rewritten) emitting at minimum session-start / phase-transition / per-test-or-challenge-start / captured-evidence-path / external-call (LLM / vision / sink-probe) / error / per-item-verdict events; (2) an **atomically-rewritten status snapshot** (a single small JSON/file written via write-temp-then-rename so a reader never observes a torn write) carrying the current session/phase/item + running counters + last verdict. Verdicts MUST use the closed vocabulary PASS / FAIL / SKIP / OPERATOR-BLOCKED (per §11.4.45 status vocabulary). The conductor tails the

stream / snapshot live (`tail -f` / a dedicated monitor) so it stays in real-time sync, can §11.4.4-interrupt on a fresh defect, and never idles blindly per §11.4.94 / §11.4.97. The channel is the framework's contract with the orchestrator, not an afterthought log.

Anti-bluff for the channel itself (§11.4 / §11.4.69). The event stream + snapshot are themselves captured-evidence artefacts: a verdict event MUST carry the evidence path that backs it (per §11.4.69 `ab_pass_with_evidence`), so a PASS event with no evidence path is a §11.4 PASS-bluff at the channel layer. A status snapshot that reports PASS while the event stream shows no captured-evidence event for that item is a contradiction the conductor MUST treat as FAIL. The channel never invents progress: an item with no start-event cannot have a verdict-event.

Decoupling (§11.4.28). When the framework is an owned, project-agnostic submodule, the channel MUST stay project-neutral — the consuming project registers its own data (endpoints, app/package identifiers, sink hosts) at runtime via the framework's public API, never hardcoded into the framework. The channel format is generic; the payloads are consumer-supplied.

Classification: universal (§11.4.17) — a real-time append-only event stream + atomically-rewritten status snapshot is a platform-neutral orchestrator-sync discipline reusable by ANY project whose conductor depends on a long-running autonomous test/QA framework; the consuming project supplies the concrete channel paths and payload schema per §11.4.35.

Composes with §11.4.4 (test-interrupt-on-discovery — the live stream is what lets the conductor STOP the instant a fresh defect's FAIL event lands), §11.4.5 / §11.4.69 (every verdict event cites its captured-evidence path), §11.4.27 (no-fakes — the framework exercises the real system, the channel reports real verdicts), §11.4.28 (owned-submodule decoupling — channel + payloads stay project-agnostic), §11.4.45 (status vocabulary + sink-side evidence reuse), §11.4.52 (autonomous-validation — the channel is how an autonomous session reports without a human watching the screen), §11.4.89 (background test execution — the channel is the conductor's window into a backgrounded long test), §11.4.94 / §11.4.97 (zero-idle — tailing the live channel replaces blind polling/idle waiting).

Propagation. Propagation gate `CM-COVENANT-114-116-PROPAGATION` enforces the literal anchor `11.4.116` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-AUTONOMOUS-FRAMEWORK-SYNC-CHANNEL` (the framework emits an append-only event stream + an atomically-rewritten status snapshot, and every verdict event carries an evidence path); paired §1.1 mutation strips the evidence-path field from a verdict event → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.116 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--no-sync-channel`, `--end-of-session-report-suffices`, `--verdict-without-evidence-path`, `--torn-status-write-OK` flag exists.

---

1 1 8

2026 06 03

§ . . — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (. . -dev remediation, - -)

Short tag: `cv-ocr-pixel-oracle-fallback`.

Forensic anchor (genericised, 2026-06-03). Several apps under test (TV-Compose / leanback streaming apps, canvas-rendered players, games) return a near-empty accessibility / semantic tree — `uiautomator dump` yields a hierarchy with zero clickable nodes, so a hierarchy-driven test cannot find the play tile, cannot read the on-screen caption, cannot confirm what the user sees. The §11.4.52 rule already SKIPs UI-driven when `uiautomator dump | grep -c clickable=true == 0`. The remediation's lesson: SKIP is not the only answer — when the semantic tree is blank, the test MUST fall back to driving + asserting via the PIXELS (computer-vision template-match to locate and tap controls; ROI OCR to read on-screen text), because the pixels are what the end user actually experiences, and a hierarchy-only tool is NOT a content oracle.

The mandate. Any test that needs to drive a UI control OR assert on-screen content MUST NOT assume the accessibility / semantic / DOM hierarchy is the source of truth for what the user sees. When the hierarchy is blank, partial, or known-unreliable for the app under test (TV-Compose, leanback, canvas/ `SurfaceView` /GL, games, custom-rendered UIs), the test MUST fall back to a **pixel oracle**: (1) **drive** input by computer-vision template-match (locate a control by its rendered appearance, then tap its screen coordinates) rather than by a hierarchy node id; (2) **assert** on-screen content by ROI OCR (read the actual rendered text in the region the user reads — caption strip, title, error overlay) with a per-word confidence floor + region-of-interest per §11.4.107(12), never by a hierarchy text attribute that may be absent or stale. The test tool MUST be chosen for its ability to BOTH drive input AND read pixels — a hierarchy-only tool cannot serve as a content oracle for these UIs. The pixel path is the §11.4.52 "near-empty hierarchy proves UI-driven INFEASIBLE" fallback made constructive: not SKIP, but pixel-drive.

Anti-bluff for the pixel oracle (§11.4.107). The CV/OCR analyzer is itself a potential bluff surface and MUST be self-validated per §11.4.107(10): a golden-good fixture (the control present / the expected caption rendered) MUST PASS and a golden-bad fixture (control absent / wrong-or-empty caption) MUST FAIL, wired into the meta-test. Thresholds (match confidence, OCR confidence floor) MUST be calibrated on the project's own captured frames, not hardcoded from literature (§11.4.107(13) / §11.4.6). An OCR PASS that cannot also FAIL its golden-bad fixture is a §11.4 bluff gate.

Honest boundary (§11.4.6). When BOTH the hierarchy is blank AND the pixel oracle is infeasible (e.g. the target output is a protected/secure surface that captures black per §11.4.112, or the content is geo-unreachable), the test SKIPs-with-reason per §11.4.3 (`topology_unsupported` / `operator_attended` / `geo_restricted`), never a fake PASS, and the operator-attended path is a tracked migration item per §11.4.52, not the permanent answer.

Classification: universal (§11.4.17) — driving + asserting via pixels when the semantic tree is blank is a platform-neutral test-oracle discipline reusable by ANY project testing custom-rendered / canvas / game / Compose / leanback UIs; the consuming project supplies its concrete CV/OCR toolchain (template-match library, OCR engine), capture mechanism, and calibrated thresholds per §11.4.35.

Composes with §11.4.5 (captured-evidence quality — OCR overlay census), §11.4.6 (no-guessing — thresholds calibrated, not assumed), §11.4.48 (UI-driven — pixel-drive is the fallback when the hierarchy path is infeasible), §11.4.49 (dual-approach — pixel-drive UI variant complements the intent variant), §11.4.52 (autonomous-validation — strict refinement: near-empty hierarchy → pixel-drive, not only SKIP), §11.4.107 (AV/test-validation — the OCR/CV analyzer is self-validated golden-good/golden-bad, ROI + confidence floor; §11.4.117 is its UI-driving companion), §11.4.112 (structural-impossibility — secure-surface black-capture is the honest boundary where even pixel-drive cannot read content).

Propagation. Propagation gate `CM-COVENANT-114-117-PROPAGATION` enforces the literal anchor `11.4.117` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-CV-OCR-PIXEL-ORACLE-FALLBACK` (a test targeting a known-blank-hierarchy app drives + asserts via a self-validated CV/OCR pixel oracle, not a hierarchy-text attribute); paired §1.1 mutation makes the pixel-OCR analyzer PASS its golden-bad fixture → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.117 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--hierarchy-is-content-oracle`, `--skip-pixel-fallback`, `--unvalidated-ocr-OK`, `--hardcoded-ocr-threshold-OK` flag exists.

---

§ . . — Discovery-pressure to confirm known-issue-set completeness mandate (. . -dev remediation, - -)

Short tag: `discovery-pressure-completeness`.

Forensic anchor (genericised, 2026-06-03). A remediation cycle has a list of reported defects. Fixing exactly those is NECESSARY but NOT SUFFICIENT — the reported set is what the operator happened to notice, biased by what they tested ("we only see what we test"). The remediation ran parallel discovery + stress streams across ALL target devices, exercising subsystems the reported defects did NOT touch, to CONFIRM the reported set IS the complete critical set (and to

surface any unreported defect before the user does). The discovery is anti-bluff only if it provides PROVABLE coverage: a list of the subsystems / journeys actually exercised, each with the outcome (no-new-issue vs new-issue-filed), so "we found nothing else" is backed by "here is everything we looked at," not by "we stopped looking."

The mandate. A remediation / release cycle MUST NOT treat "every reported defect is fixed" as "the build is good." After (or in parallel with) fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices / environments that deliberately exercises subsystems, journeys, and edge cases BEYOND the reported defects — to confirm the reported set is the COMPLETE critical set and to surface unreported defects before the end user does. The discovery pass MUST produce PROVABLE coverage: an enumerated list of the subsystems / user-journeys / stress scenarios actually exercised, each with its outcome (no-new-issue, or a new tracker entry filed per §11.4.15 / §11.4.16). "We found no other issues" is a §11.4 bluff unless accompanied by "here is the enumerated set we exercised" — absence of evidence of looking is not evidence of absence. Newly-discovered defects trigger §11.4.4 test-interrupt + the §11.4.114/ §11.4.115 isolation→RED→fix loop.

Why necessary-not-sufficient. A cycle that fixes only the reported defects ships with the operator's coverage bias baked in: the subsystems the operator did not test on the broken build are still unverified on the fixed build. Discovery pressure (everyday-user-simulation journeys + chaos/stress per §11.4.85, run autonomously per §11.4.52 across every device per §11.4.119) is how the cycle earns the claim "this is the complete critical set," converting an unbounded "are there other bugs?" into a bounded, evidenced "here is what we exercised and what we found."

Honest boundary (§11.4.6). Discovery pressure reduces the unknown-unknown surface; it does not prove zero remaining defects (no finite test set can). The claim the cycle earns is "the reported set + the enumerated discovery set are all addressed," with the discovery set's coverage explicitly bounded — never "the build is bug-free." Subsystems NOT exercised are stated as such (an honest coverage gap per §11.4.3), not silently implied clean.

Classification: universal (§11.4.17) — discovery-pressure to confirm known-issue-set completeness is a platform-neutral release-confidence discipline reusable by ANY project with a remediation/release cycle; the consuming project supplies its subsystem inventory, everyday-user-journey set, stress scenarios, and target-environment matrix per §11.4.35.

Composes with §11.4.4 (test-interrupt — a discovery finding STOPs the cycle), §11.4.5 / §11.4.69 (the discovery pass captures evidence per subsystem exercised), §11.4.25 (full-automation-coverage — the discovery set is part of the coverage ledger), §11.4.40 (full-suite retest — discovery runs alongside / inside it), §11.4.42 (iteration-discipline — discovery is the smoke-then-full progression's breadth dimension), §11.4.52 (autonomous-validation — discovery journeys run without a human), §11.4.85 (stress + chaos — discovery pressure IS the stress dimension applied for completeness), §11.4.114 / §11.4.115 (a discovered defect enters the isolation→RED→fix loop), §11.4.119 (single-resource-owner — parallel per-device discovery needs the ownership partition).

Propagation. Propagation gate `CM-COVENANT-114-118-PROPAGATION` enforces the literal anchor `11.4.118` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-DISCOVERY-COVERAGE-ENUMERATED` (a remediation/release cycle records an enumerated subsystem/journey discovery-coverage list with per-item outcome, not a bare "no other issues found"); paired §1.1 mutation replaces the enumerated list with a bare no-issues claim → the gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

Canonical authority: this Constitution.md §11.4.118 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--reported-set-suffices`, `--skip-discovery-pass`, `--no-issues-without-coverage-list`, `--assume-complete` flag exists.

---

§ . . — Fix-breaks-its-own-gate reconciliation mandate (. . -dev remediation, - -)

Short tag: `gate-reconciliation-not-fake-pass` .

Forensic anchor (genericised, 2026-06-03). A correct fix removed the regression-causing behaviour (the audio fix removed the `handledByDualHdmi` short-circuit that diverted 5.1 off the multichannel thread). But an EARLIER gate had been authored to assert the OLD — now-known-broken — behaviour's presence. After the fix, that gate FAILed. The FAIL was the CORRECT signal that the fix worked: the gate was asserting a behaviour the fix deliberately removed. Two wrong responses are forbidden — fake-passing the gate (editing it to always pass), or reverting the correct fix to satisfy the stale gate. The right response is to RECONCILE the gate: rewrite it to assert the NEW mechanism (evidence-backed), so it once again guards the now-correct behaviour.

The mandate. When a correct fix causes a pre-existing gate / test to FAIL because that gate asserted the OLD (now-removed-or-changed) behaviour, the gate FAIL is the CORRECT signal that the fix landed — it MUST NOT be suppressed by either of the two forbidden responses: (1) **fake-passing** the gate — editing it to `always pass` , weakening its assertion to a tautology, or deleting it to make the FAIL disappear (a §11.4 bluff at the gate layer, and a §11.4.84 mutation-residue risk); (2) **reverting the correct fix** to satisfy the stale gate (re-introducing the defect to keep a now-wrong gate green). The required response is **RECONCILIATION**: rewrite the gate so it asserts the NEW mechanism the fix introduced, backed by captured evidence of the new correct behaviour, so the gate once again guards the right invariant. After reconciliation, the gate's paired §1.1 mutation MUST be updated too (the mutation must break the NEW invariant). The reconciliation MUST be a visible, evidence-cited change — "rewrote gate X to assert because fix Y removed ; new captured evidence: " — never a silent assertion-weakening.

Distinguish reconciliation from bluffing. Reconciliation rewrites the gate to assert a REAL new invariant (the fix's intended mechanism), proven by captured evidence, with the paired mutation updated to break it. Bluffing weakens the gate so it can no longer FAIL on the defect class it was meant to catch. The discriminator: after reconciliation the gate + its mutation still form a valid §1.1 pair

(mutate the new invariant → gate FAILs); a bluffed gate's mutation no longer makes it FAIL (the assertion became a tautology). A reconciled gate is provably still a guard; a bluffed gate is provably blind.

Honest boundary (§11.4.6). A gate FAIL after a fix is NOT automatically "the gate is stale, reconcile it." It MUST first be investigated per §11.4.102 — the FAIL may be the gate correctly catching a REGRESSION the fix introduced (the fix broke something else), in which case the FIX is wrong, not the gate. Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour AND the new behaviour is the intended, evidence-confirmed mechanism. "The gate is just stale" without that investigation is a §11.4.6 guess that can mask a real regression.

Classification: universal (§11.4.17) — reconcile-the-gate-don't-fake-pass-and-don't-revert is a platform-neutral fix-vs-gate-conflict discipline reusable by ANY project with regression gates; the consuming project supplies its gate framework and paired-mutation mechanism per §11.4.35.

Composes with §11.4.1 (FAIL-bluffs — fake-passing the gate is the inverse bluff), §11.4.4 (test-interrupt — a post-fix gate FAIL triggers investigation before reconcile), §11.4.6 (no-guessing — "stale gate" is a hypothesis until investigation proves it), §11.4.84 (working-tree quiescence — a gate edited to `always pass` is mutation residue that must never ship), §11.4.102 (systematic-debugging — the mandatory investigation that distinguishes stale-gate from new-regression), §11.4.108 (four-layer fix-verification — the reconciled gate asserts the fix's runtime signature / new mechanism), §1.1 (paired mutation — the reconciled gate's mutation must break the NEW invariant, re-proving it is a guard not a tautology).

Propagation. Propagation gate `CM-COVENANT-114-120-PROPAGATION` enforces the literal anchor `11.4.120` across the consumer fleet; paired §1.1 meta-test mutation strips the literal → the gate FAILs. Recommended per-family gate `CM-GATE-RECONCILED-NOT-FAKE-PASSED` (when a fix changes a behaviour a gate asserted, the gate's reconciliation is an evidence-cited rewrite whose paired mutation still makes it FAIL — never an assertion weakened to a tautology); paired §1.1 mutation weakens a reconciled gate to `always pass` and asserts its mutation no longer makes it FAIL → the meta-gate FAILs. (Gate-code implementation lands as a separate work item; this anchor defines the contract.)

GATE SEAM-PLACEMENT — a gate whose precondition the gated work itself produces is on the WRONG SEAM (extension, research-derived, 2026-07-17). §11.4.120's trigger is a gate that FAILs — i.e. a gate that RAN. Its sibling case is a gate that CANNOT RUN in the ordering imposed on it, and the required response is different: not reconciliation, but re-seating the gate at the seam where it is satisfiable. Forensic FACT (genericised): work was ordered "run checks E1-E4, then implement P1 **only if** E1-E4 pass" — but the gate's own text defined E2 as running **on a P1-gated build**, so E2 could not exist before P1 existed; the order was unexecutable by construction. The source of truth, read exactly, gated something else entirely: building behind a default-off flag was permitted, and it was **ENABLING** the flag that required E1-E4 — the gate was on ACTIVATION, and had been imposed on AUTHORSHIP. Therefore: **(a) EXECUTABILITY IS CHECKED BEFORE A GATE IS IMPOSED** — if gate G requires artifact A, and A is produced by the very work G gates, then G is unsatisfiable at that seam and belongs at a LATER one (activation / enablement / release), NEVER at authorship; imposing it at authorship makes the work impossible to start and the gate impossible to pass, which is not rigor, it is a deadlock. **(b) RE-READ THE SOURCE OF TRUTH BEFORE RE-SEATING** — the circularity is usually an ARTIFACT OF THE ORDER, not of the gate: the gate's own text names its real seam, and that text governs over any restatement of it (a restatement that moved the gate from "enabling" to "writing" is a §11.4.6 misstatement of the requirement, and the fix is to obey the source, not to weaken the gate). **(c) THE FORBIDDEN RESPONSES ARE §11.4.120's OWN** — an unsatisfiable gate MUST NOT be fake-passed, deleted, weakened to a tautology, nor improvised past by building the artifact and retro-declaring the gate met; and the gated work MUST NOT be abandoned as "blocked" when the gate simply sits on the wrong seam. **Honest boundary (§11.4.6):** an unrunnable gate is NOT automatically mis-seated — investigate per §11.4.102 first; it may be correctly seated and its precondition genuinely missing, which is the §11.4.69 `artifact_not_yet_built` NOT-YET-RUNNABLE state (§11.4.135 seam-dependent), not a seam defect. Reporting the circularity is mandatory and already required by §11.4.6 / §11.4.66 / §11.4.101 / §11.4.201(4) — this clause adds the seam-placement rule those anchors do not carry. §11.4.110's clash detector cannot catch this class: its input is a change DIFF, and an ordering instruction is not a diff.

Gate contract for the seam-placement clause (extension, 2026-07-17). `CM-GATE-RECONCILED-NOT-FAKE-PASSED` is EXTENDED: an imposed gate whose required precondition artifact is produced by the very work it gates → FAIL (the gate

is mis-seated and MUST be re-seated at activation/enablement/release), and the FAIL names the circularity rather than the work. Paired §1.1 mutation: impose an authorship-seam gate whose precondition only the gated work can produce → the gate FAILs; re-seat it at activation → PASSes. Its golden-FALSE fixture MUST include a correctly-seated gate whose precondition is merely not-yet-built (the §11.4.69 `artifact_not_yet_built` state) — the gate MUST NOT fire on that, or it becomes the §11.4.201(1) false positive that condemns a healthy gate. Gate-code = a separate work item; this is the contract, not a claim the code has shipped (§11.4.6).

Canonical authority: this Constitution.md §11.4.120 in the HelixConstitution submodule. All consuming projects restate + cite via §11.4.35 inheritance.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--fake-pass-stale-gate`, `--revert-fix-for-gate`, `--weaken-assertion-to-pass`, `--delete-failing-gate`, `--impose-unsatisfiable-gate`, `--improvise-past-circular-order`, `--gate-authorship-on-its-own-output` flag exists.

2026 06 08

§ . . — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User mandate, - -)

Forensic anchor (the canonical case this anchor exists to prevent, FACT): the wrong-subtitle-on-2nd-display defect was "fixed" in a prior release via a source-side `CONTROL_MENU_LABEL_DENYLIST` that NO test mirrored or re-ran, so the NEXT chrome class (a settings-menu label) recurred silently while the GREEN suite passed.

Every project MUST maintain a STANDING regression-guard suite that runs on EVERY build+deploy and BLOCKS the release tag on any failure. Every closed defect (the project's stable ticket id, e.g. ATM-NNN) MUST, in the SAME commit as its fix (extending the §11.4.43 DOCUMENT step), register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite — `RED_MODE=1` capturing the historical defect on a pre-fix artifact (the proof the guard is real), `RED_MODE=0` the standing GREEN guard asserting the defect is ABSENT. A closure without a

registered guard is a §11.4.123 violation (closure without permanent regression proof). The suite runs FIRST in the post-deploy cycle (highest-risk set per §11.4.132 risk-ordering) and is a §11.4.40 release-gate blocker. This is industry-standard bug-driven testing (cite: Google for Developers content-driven testing https://developers.google.com/solutions/content-driven/backend/testing; AOSP CTS/Tradefed continuous-regression model https://source.android.com/docs/compatibility/cts) made mechanical + enforced.

Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.40 / §11.4.43 / §11.4.46 / §11.4.50 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.124 / §11.4.130 / §11.4.132. Recommended gates `CM-REGRESSION-GUARD-REGISTERED` + `CM-REGRESSION-GUARD-SUITE-WIRED` + `CM-COVENANT-114-135-PROPAGATION` (literal `11.4.135`) + paired §1.1 meta-test mutations (gate-code = separate work item).

Verdict-coverage at the release seam — ABSENCE of a verdict blocks exactly as a FAIL does (extension, research-derived, 2026-07-17). Forensic FACT (genericised, 2026-07-17): a consuming project's standing guard suite computed its exit code from ONLY its FAIL count — a guard that was never run (fix never built/deployed → REQUIRES-REBUILD SKIP → PENDING) affected nothing; measured live on a release-candidate target, 61 registered guards produced PASS 0 / FAIL 5 / PENDING 56 and the suite exited 0 "not blocked", and the release-tag tooling consulted the guard suite ZERO times — to the release gate, "never verified" and "verified good" were the same colour. That single fail-open explains, without needing hundreds of separate bugs, how 95% of that tracker's done-claiming items shipped unguarded and how the recorded reopen-per-recorded-fix rate reached 52%. Therefore (ALL hold): (1) **the suite's exit semantics MUST incorporate COVERAGE, never only the FAIL count** — a PENDING/never-run verdict is not a pass AND not ignorable at the seams that ship; (2) **the release seam blocks on verdict coverage of the release-candidate artifact** — the tag is refused unless, for the candidate's artifact fingerprint, every registered guard whose required topology is present on at least one release-validation target has produced a real PASS-or-FAIL verdict, and zero FAILs: `uncovered = registered ∧ topology-present ∧ no-verdict-for-this-fingerprint`; refuse while `uncovered ≠ ∅ ∨ N_FAIL > 0`; (3) **the release-tag tooling MUST consult the suite/verdict store mechanically** — a release path that never reads the guards is itself a §11.4 bluff surface regardless of how green the suite runs elsewhere; (4) **the two legitimate PENDING classes are handled without a fail-open AND without**

a fail-closed wall: on intermediate artifacts a REQUIRES-REBUILD PENDING is honest and non-blocking, but on the release candidate it is a contradiction — either the wrong artifact is deployed (re-run on the candidate) or the fix genuinely missed the build (then its item MUST NOT be in the release's done-set); both resolutions are forced, neither is a permanent exemption. Topology-absent guards are exempt from coverage ONLY via a checked-in target-pool topology map and MUST be enumerated in the release changelog as honest coverage gaps (§11.4.3 / §11.4.118) — never a silent exemption; a naive "PENDING always blocks" rule is REJECTED as a §11.4.201 FAIL-bluff (it would block every honest intermediate run and teach operators to disable the gate); (5) **adoption ratchet** — a pre-existing unguarded done-claiming backlog is snapshotted ONCE into a committed ratchet file and may only shrink (the live unguarded set must be a subset of the snapshot AND the snapshot may not gain lines vs its committed version), so day one is green, the disease cannot spread, and every reopened/re-touched item exits the ratchet permanently. Recommended gate `CM-RELEASE-VERDICT-COVERAGE` — self-tested per §11.4.205 / §11.4.107(10): golden-bad fixture = a topology-present guard with no verdict for the candidate fingerprint → the release gate MUST FAIL; golden-good = a complete verdict set → MUST PASS; negative control = a properly-enumerated topology-absent guard → MUST PASS (the false-positive guard) — + paired §1.1 mutation (revert the exit semantics to FAIL-count-only, or de-wire the release-tag consultation → the gate FAILs; gate-code = separate work item). Composes §11.4.3 / §11.4.40 / §11.4.107(10) / §11.4.115(F) / §11.4.118 / §11.4.146(D3) / §11.4.201 / §11.4.205.

Canonical authority: constitution submodule `Constitution.md` §11.4.135. Non-compliance is a release blocker. No escape hatch — no `--skip-regression-guard`, `--no-guard-on-close`, `--guard-optional` flag — and, per the 2026-07-17 extension, no `--pending-does-not-block-the-candidate`, `--fail-count-only-exit`, `--release-without-consulting-guards`, `--silent-topology-exemption` flag.

§ . . — Real-content end-to-end playback-test mandate (User mandate, - -)

Refines/strengthens §11.4.107. Any test asserting media playback works MUST drive REAL content (catalog stream or offline reference clip) through the user's path (§11.4.48 UI-driven → §11.4.117 CV/OCR fallback) and assert it genuinely PLAYS via the §11.4.107 liveness battery PLUS a decoder-health census — a numeric drop-buffer budget, no buffer-timestamp re-order/discard, no codec-reject (cite: Android/Media3 ExoPlayer OEM pre-OTA playback-test mandate https://developer.android.com/media/media3/exoplayer/oems — "too many dropped buffers" >25, "unexpected presentation timestamp", "test timed out" failure modes). Metadata-only / launch-only / registration-only / single-frame / config-only PASS is forbidden (§11.4 / §11.4.1). A golden/reference clip corpus (cite: BBC ExoPlayer testing samples https://github.com/bbc/exoplayer-testing-samples; USPTO 8,387,015 empirical media-player testing) is the offline ground-truth.

Classification: universal (§11.4.17). Composes §11.4.5 / §11.4.48 / §11.4.50 / §11.4.107 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Recommended gate `CM-REAL-CONTENT-PLAYBACK-TEST` + `CM-COVENANT-114-136-PROPAGATION` (literal `11.4.136`) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.136. Non-compliance is a release blocker. No escape hatch — no `--launch-proves-playback` (11 4 137), `--skip-decoder-health`, `--metadata-playback-pass-suffices` flag.

2026 06 08

§ . . — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandate, - -)

Refines §11.4.117 (pixel-oracle) + §11.4.107 (liveness) + §11.4.112 (structural-impossibility).

Forensic anchor (FACT): tests tasked to "physically verify the 2nd-display (or 1st where no TV) subtitle" PASSed GREEN while subtitles did NOT show / showed WRONG, because (a) the streaming app's player surface is FLAG_SECURE so `screencap -d <secondary>` returns BLACK (cite: Android Developers https://

developer.android.com/security/fraud-prevention/activities; Nightwatch https://wwws.nightwatchcybersecurity.com/2016/04/13/research-securing-android-applications-from-screen-capture/; WithSecure https://labs.withsecure.com/advisories/screencapture-via-ui-overlays-in-mediaprojection) → autonomous PIXEL verification is structurally impossible (§11.4.112), so the test fell back to a non-pixel proxy (the accessibility-scraped forwarded cue / `persist.atmosphere.subdebug` logcat), and (b) the proxy's validation accepted a chrome/menu LABEL (`Аудио и субтитры`) as a valid subtitle because the prose floor accepted any multibyte prose and NO menu-label denylist + NO position/cadence check existed.

The mandate: a test asserting a subtitle/caption is correct MUST classify the *content class* of the rendered/forwarded cue — a present cue is NOT a correct cue. A cue is CHROME (FAIL) if it is a known control/menu label (a closed multilingual deny-list MIRRORED from the source-side denylist, case-folded both sides incl. non-ASCII scripts), is time-counter/numeric chrome, is NOT prose, is OUTSIDE the lower safe-title position band where caption position is available (cite CEA-708 9-anchor caption-window grid: captions live in the lower-center safe band, chrome elsewhere https://www.gte-media.com/cea-608-and-cea-708-captions/), OR is STATIC across the window (a real subtitle changes → dialogue cadence; a menu label does not → ≥2 distinct prose cues required, a metamorphic relation per https://en.wikipedia.org/wiki/Metamorphic_testing). A cue is DIALOGUE (PASS) only when prose, not-denied, not-chrome, position-ok, AND cadence ≥2 OR it fuzzy-matches the SOURCE-extracted expected cue via normalized edit distance (§11.4.123 host-side ground truth; cite SubER https://github.com/apptek/SubER). The content oracle MUST be self-validated golden-good/golden-bad (§11.4.107(10)) and the test-side deny-list MUST be verified present in the SHIPPED artifact (§11.4.108) — a source-green denylist with no test mirror + no artifact check is the exact recurrence pattern this anchor forbids.

Secure-display-proxy honesty (§11.4.112): where FLAG_SECURE makes pixel verification structurally impossible, the rock-solid autonomous proof is the player's own caption telemetry (the subdebug/Cue ground-truth channel) + source-track presence + content-class oracle — NEVER a faked pixel "physical" pass; the human-eye pixel confirmation is `operator_attended` (§11.4.52) with a tracked migration item. App-agnostic (keys off content class, not per-app rule) so it is the standing guard for the whole roster.

Classification: universal (§11.4.17). Composes §11.4.3 / §11.4.5 / §11.4.6 / §11.4.107 / §11.4.108 / §11.4.112 / §11.4.115 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Recommended gate `CM-SUBTITLE-CONTENT-CORRECTNESS-ORACLE` + `CM-COVENANT-114-137-PROPAGATION` (literal `11.4.137`) + paired §1.1 meta-test mutation (strip the denylist/position/cadence check → golden-bad `Аудио и субтитры` PASSes → gate FAILs).

Canonical authority: constitution submodule `Constitution.md` §11.4.137. Non-compliance is a release blocker. No escape hatch — no `--present-cue-is-correct`, `--skip-chrome-oracle`, `--length-heuristic-suffices`, `--pixel-pass-on-secure-display`, `--skip-position-check`, `--skip-cadence-check` flag.

11 4 138

2026 06 08

§ . . — Operator-escape => mandatory bluff-audit + permanent guard (User mandate, - -)

When the operator (or any out-of-band channel) finds a defect that the GREEN test suite passed, this is by definition a §11.4 PASS-bluff — it MUST trigger, before the fix is closed: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause; (2) a bluff-audit identifying the EXACT assertion that should have caught it but didn't, cited to `file:line` (canonical example: `lib/subtitle_content_validation.sh:sub_is_prose()` returning TRUE for `Аудио и субтитры`); (3) a permanent §11.4.135 regression guard registered in the SAME commit as the fix, with its §11.4.115 RED capturing the operator-found defect; (4) the bluff-audit committed under `docs/research/<scope>/<defect>_bluff_audit/`. Closing an operator-found defect WITHOUT the bluff-audit + permanent guard is itself a §11.4 violation (the bluff that let it through is still live and the defect will recur — the operator's "repeatedly reworking + reopening the same issues with no real progress" pain).

Classification: universal (§11.4.17). Composes §11.4 / §11.4.1 / §11.4.102 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.135. Recommended gate `CM-OPERATOR-ESCAPE-BLUFF-AUDIT` + `CM-COVENANT-114-138-PROPAGATION` (literal `11.4.138`) + paired §1.1 meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.138. Non-compliance is a release blocker. No escape hatch — no `--close-without-bluff-audit`, `--operator-find-is-just-a-bug`, `--skip-permanent-guard` flag.

§ . . — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-10):

> "All video streaming apps ... require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that's it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + a consuming project's extensions."

Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app's OWN user interface — launch the app → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play / Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / `am start -a VIEW` / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app's own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator actually cares about — "the show I picked actually plays" — unproven. Such shortcuts exercise neither the catalog/auth/title-resolution path nor the real Play-button → decoder → routing transition where the end-user defects live (the operator's "we just eventually play on 2nd display sample, and that's it mostly" is exactly this gap).

The mandate (ALL must hold): (1) **Real journey, not a shortcut** — the test drives launch → catalog browse → specific-title selection → real Play/Resume press through the app's own UI (§11.4.48 UI-driven), NEVER a deep-link / intent /

sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target), via the §11.4.107 liveness battery (live advancing frames + frame-advance counter + not-stale-from-previous) on the §11.4.136 real-content path, plus the §11.4.137 subtitle content-correctness oracle for the chosen title's captions (a present cue is NOT a correct cue); (3) **Non-introspectable UIs use the pixel oracle** — when the app's accessibility/semantic hierarchy is blank or unreliable (TV-Compose / leanback / canvas / GL streaming UIs), the test DRIVES input + ASSERTS content via the §11.4.117 CV/OCR pixel oracle (template-match the Play control + ROI-OCR the title/caption), never a hierarchy-only tool; (4) **Login via the credential single source** — apps requiring sign-in authenticate from the project's credential single-source-of-truth (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is `operator_attended` SKIP-with-reason per §11.4.52 + §11.4.3 with a tracked migration item (§11.4.15 / §11.4.16) — NEVER a metadata-only / sample-played / routing-only PASS. Honest boundary (§11.4.6): "the routing fired so the title is playing" is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it; a SKIP for genuine infeasibility is correct, a PASS that never pressed the real Play button on the real title is the bluff this anchor forbids.

Classification: universal (§11.4.17) — a platform-neutral end-user-journey discipline reusable by ANY project that ships or integrates video-streaming applications; the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Composes §11.4.48 (UI-driven traversal of the real journey) / §11.4.107 (liveness — the chosen content actually advances) / §11.4.117 (CV/OCR pixel oracle for non-introspectable streaming UIs) / §11.4.136 (real-content end-to-end playback) / §11.4.137 (subtitle content-correctness for the chosen title) / §11.4.52 (autonomous-first; operator-attended only when genuinely infeasible) / §11.4.3 (topology / geo SKIP-with-reason) / §107 (end-user usability is the bar) / §1.1. Propagation gate `CM-COVENANT-114-143-PROPAGATION` (literal `11.4.143` across the consumer fleet) + recommended gate `CM-VIDEO-REAL-JOURNEY-TEST` (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPs-with-reason) + paired §1.1 meta-test mutation (replace

a real-journey test's browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.143. Non-compliance is a release blocker regardless of context. No escape hatch — no `--sample-playback-ok` , `--skip-real-journey` , `--deep-link-suffices` , `--routing-only-pass` , `--no-title-selection` flag exists.

§ . . — Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-06-25):

> "MANDATORY test types EACH project MUST have with coverage aiming to 100% wihout false results, rock solid evidence produced covered with hard physical proofs and zero bluff: unit, integration, e2e, full automation, challenges (challenges submodule), helix qa (helix qa submodule with proper tests banks (suites) and comprehensive fully autonomous qa sessions), ddos tests, security tests, stress and chaos tests, concurrency (atomicy) tests, race conditions or deadlocks tests, memory tests, bechmarking tests."

Every project under this Constitution MUST be covered by the following CLOSED, ENUMERATED set of test types, each driven to **as close to 100% coverage as the domain permits**, with every PASS backed by **rock-solid captured PHYSICAL evidence** (§11.4.5 / §11.4.69 / §11.4.107) and **zero false results, zero bluff of any kind** (§11.4 / §11.4.1 / §11.4.6). A green result with no falsifiable captured evidence is a §11.4 PASS-bluff regardless of which layer reports it. §11.4.169 is the strict, explicitly-enumerated expansion of §11.4.27 (no-fakes-beyond-unit + 100%-test-type coverage) — it freezes the canonical test-type SET and binds each type to captured evidence and, where named, to the owned submodule that supplies it.

The mandatory test-type set (each REQUIRED where the domain warrants it; an honest §11.4.3 SKIP-with-reason — topology / hardware / credential genuinely absent — is the ONLY permitted absence, NEVER a silent gap):

1. **Unit** — isolated logic; the ONLY layer where mocks / stubs / fakes / placeholders are permitted (§11.4.27(A)).
2. **Integration** — real, fully-wired components against real infrastructure (no fakes beyond unit), infra booted on-demand via the **containers** submodule (§11.4.76 / §11.4.161 rootless).
3. **End-to-end (e2e)** — the full user journey across the real System, captured-evidence per §11.4.5 / §11.4.107.
4. **Full-automation** — fully autonomous, re-runnable end-to-end with NO manual intervention after start (§11.4.25 / §11.4.52 / §11.4.98), deterministic across N iterations (§11.4.50).
5. **Challenges** — the **challenges** submodule (`vasic-digital/challenges`): anti-bluff Challenge banks that score PASS only on positive captured evidence (§11.4.27(B) / §11.4.4(b) layer 4).
6. **HelixQA** — the **helix_qa** submodule with proper written **test banks (suites)** for every application / service / platform AND comprehensive **fully-autonomous QA sessions** driving every registered bank with captured wire evidence per check (§11.4.27).
7. **DDoS / load-flood** — sustained adversarial traffic; the System refuses cleanly OR degrades gracefully, never collapses; captured throughput / latency / error-rate evidence.
8. **Security** — authn / authz, injection / taint, secret-leak (§11.4.10), transport + crypto, dependency CVEs, default-deny enforcement; composes the **security** submodule.
9. **Stress + Chaos** — sustained load + concurrent contention + boundary inputs + failure-injection (process-death / network-fault / input-corruption / resource-exhaustion / state-corruption) with categorised recovery + captured evidence (§11.4.85).
10. **Concurrency / atomicity** — correctness under concurrent callers; no lost updates; transactional atomicity proven; idempotency under replay.

11. **Race-condition / deadlock** — race detector + lock-order / deadlock analysis under contention; no blocking operation inside a shared-lock region (the §1 development principles); captured detector output is the evidence.
12. **Memory** — leak census over soak, peak-RSS ceilings (e.g. the iOS-NEPacketTunnelProvider-class budget), allocation / fragmentation profile; no unbounded growth across a 24h / N-iteration soak.
13. **Benchmarking / performance** — p50 / p95 / p99 latency + throughput + resource cost vs a recorded baseline; a regression vs baseline is a finding, not noise.

Four-layer enforcement per §11.4.4(b) of the suite itself: pre-build gate (each required type present + executable + parseable for the domain), post-build, on-device / runtime where applicable, and a paired §1.1 meta-test mutation per type (strip a type's evidence-capture or its anti-bluff assertion → the gate FAILs). The per-project coverage ledger (§11.4.25 / §11.4.52) MUST classify every feature × test-type × evidence-state; a missing required type, or an `OPERATOR_ATTENDED_ONLY` row, is a release blocker until promoted or honestly SKIP-justified (§11.4.3 / §11.4.52).

Composes / strengthens §11.4.4 / §11.4.5 / §11.4.6 / §11.4.25 / §11.4.27 (its strict enumerated expansion) / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.76 / §11.4.85 / §11.4.98 / §11.4.107 / §11.4.123 / §11.4.135 / §11.4.146 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete per-type harnesses, infra, and calibrated thresholds per §11.4.35.

Pre-build gate `CM-COVENANT-114-169-PROPAGATION` (literal `11.4.169` across the consumer fleet) + recommended gate `CM-MANDATORY-TEST-TYPES-COVERED` (every required test type present-or-honestly-SKIPPED per feature, each PASS citing captured evidence) + paired §1.1 meta-test mutation (strip a required type or drop its evidence citation → gate FAILs; gate-code = separate work item).

EXTENSION — SUITE SHAPE: the shape is a consequence, and the generating rule is push-tests-down (2026-08-20). §11.4.169 freezes WHICH KINDS of test must exist; it says nothing about WHERE a given behaviour should be tested, and a project can satisfy every enumerated type while placing its assertions at an altitude that proves little. The AI-curriculum corpus (module 30) supplies the placement rule and — importantly — REFUSES to prescribe a shape, which this extension preserves rather than overrides. **(a) NO SHAPE IS**

MANDATED. Corpus: *"the shape is a consequence, not a goal."* The two named shapes are the **test pyramid** (*"many fast unit tests at the base, fewer integration/service tests, very few slow end-to-end tests … ideal when most of your risk is in pure logic"*) and the **testing trophy** (*"a static base (types + lint), a fat integration middle, a thin end-to-end top … for I/O-heavy web/backend code, integration tests hit the best confidence-per-cost because they exercise the real wiring — DB, queue, provider — where the actual bugs live"*). The selection criterion is a single question — *"where do I get the most confidence per second?"* — answered from *"where this system's bugs live, not dogma."* The corpus explicitly marks "the trophy is always correct for every system" as a WRONG answer, and states NO numeric layer ratios; a project adopting a distribution supplies it as DATA per §11.4.35 (the corpus's own vocabulary is qualitative: "many / fewer / very few", "static base / fat middle / thin top"). **(b) THE GENERATING RULE IS MANDATED.** Corpus (Fowler): *"push each test as far down as it will still catch the bug; if a high-level test fails with no lower-level test failing, add the lower-level one and consider deleting the redundant high-level one."* This is the operative discipline §11.4.169 was missing: a behaviour is tested at the LOWEST altitude that still catches its defect, and a high-level failure with no lower-level counterpart is a coverage signal to be acted on, not merely a red build. **(c) THE TWO NAMED ANTI-PATTERNS, ONE AT EACH END.** The **ice-cream cone** — *"End-to-end tests give the most confidence, so we have lots of them" → "slow, flaky, still misses unit edges"* — is a §11.4.169 violation at the top; and **mock-heavy unit theater** — *"mock the DB, mock the provider, mock the collaborators, assert the code calls the mocks as scripted — and the suite passes precisely because it never touched the system it claims to test"*, which *"can hit 100% coverage (every line ran against a mock) and still catch nothing"* — is a violation at the bottom, already forbidden by §11.4.27(A) (mocks permitted ONLY in unit tests) and by §11.4.224(C) (an assertion-free test raises coverage identically to a proving one). The corpus names its three sub-mechanisms: the mock *"encodes a belief that drifts"*; it *"tests interaction, not effect"* (*"'The code called charge() once' is not 'a charge happened and the ledger balanced'"*); and it *"couples tests to implementation"* so a behaviour-preserving refactor breaks it. Honest boundary (§11.4.6): this extension mandates a DECLARED, JUSTIFIED shape and the push-down rule — it does NOT mandate any particular shape, any ratio, or any per-layer count, and a project whose bugs genuinely live in the wiring is CORRECT to run a fat integration middle. Recommended mechanism gate `CM-SUITE-SHAPE-DECLARED-AND-PUSHED-DOWN` (the project declares its chosen shape with the risk-location justification that selected it; a high-level test that fails with no lower-level

counterpart opens a tracked §11.4.197 item to add the lower-level test) + paired §1.1 mutation (remove the shape declaration, or record a high-level-only failure with no tracked lower-level follow-up → the gate MUST FAIL; golden-FALSE per §11.4.201(1): a declared trophy shape on a genuinely I/O-heavy system MUST NOT fire it merely for having few unit tests). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.169. Non-compliance is a release blocker regardless of context. No escape hatch — no `--skip-test-type`, `--partial-coverage-ok`, `--evidence-optional`, `--challenges-not-applicable`, `--skip-helixqa`, `--no-chaos`, `--no-ddos`, `--skip-memory-test`, `--skip-race-detector`, `--bench-optional` flag exists.

> **[COLLISION-REDIRECT 2026-08-18]** The 2026-06-25 HelixTranslate canonical translation pipeline mandate previously carried §11.4.140 here has been re-minted to **§11.4.255** to resolve the pre-existing double-mandate collision surfaced by §11.4.227(B) block-integrity audit. The other §11.4.140 mandate (Universal action-prefix system, 2026-06-09) at line ~9114 is unchanged and remains the sole occupant of the §11.4.140 anchor number. Operator-approved re-mint via AskUserQuestion on 2026-08-15.

> **[COLLISION-REDIRECT 2026-08-18]** The 2026-06-25 Mandatory independent per-language translation review mandate previously carried §11.4.141 here has been re-minted to **§11.4.256** to resolve the pre-existing double-mandate collision surfaced by §11.4.227(B) block-integrity audit. The other §11.4.141 mandate (Token-efficiency, 2026-06-09) at line ~9233 is unchanged and remains the sole occupant of the §11.4.141 anchor number. Operator-approved re-mint via AskUserQuestion on 2026-08-15.

---

§11.4.189 — Most-reopened cases get extra-depth live-testing scrutiny FIRST (User mandate, 2026-07-10). Verbatim operator mandate: "All live testing MUST pay extra attention to cases which have been reopened numerous times and those cases in depth retested, investigated, fully validated and verified with real physical results and no bluff of any kind anywhere!" All LIVE TESTING (on-device / on-target runtime validation, deploy-and-observe) MUST give EXTRA-DEPTH retest + in-depth investigation + FULL validation/verification with REAL PHYSICAL captured evidence (§11.4.5/§11.4.69/§11.4.107 — captured audio/video/sysfs/dumpsys/sink-side/runtime-signature) and NO bluff of any kind ANYWHERE to the cases that have been REOPENED THE MOST TIMES (highest §11.4.55 reopens-count) — the empirically-most-fragile set gets the DEEPEST live scrutiny, and it gets it FIRST. (1) **WHAT** — for the highest-reopens-count cases, live testing is NOT a single pass: it re-runs the case at EXTRA DEPTH (more iterations per §11.4.50 deterministic consistency, wider edge-case + regression coverage), RE-INVESTIGATES the underlying defect per §11.4.102 systematic-debugging + §11.4.146 reproduce-first, and CONFIRMS with rock-solid captured physical evidence per §11.4.123 — never metadata-only / config-only / absence-of-error / grep-without-runtime (§11.4/§11.4.1), never a false result. (2) **ORDER** — the most-reopened set is scrutinised FIRST, ahead of the rest of the live-test suite (the §11.4.132 risk-descending discipline, with most-reopened the load-bearing key). (3) **KEY** — priority is keyed off the §11.4.55 reopens-count (the consuming project supplies its reopens-count source per §11.4.35, e.g. the §11.4.93 workable-items DB `reopens_count`); a high reopens-count is the strongest empirical fragility signal (§11.4.132(d)). (4) **PROOF** — each most-reopened case's live PASS cites its captured-evidence artefact path (§11.4.69/§11.4.107) on a CLEAN/fresh deployment (§11.4.108 runtime-signature), with the §11.4.115 RED→GREEN polarity flip captured where a permanent regression guard exists (§11.4.135). STRENGTHENS/REFINES §11.4.132 (risk-ordered validation factor (d) most-reopened — §11.4.189 promotes most-reopened into its OWN EXTRA-DEPTH mandate at the LIVE-TEST layer, not merely one ranking factor among four) + §11.4.55 (reopens-count as the priority key) + §11.4.129/§11.4.130 (validate-the-just-fixed / highest-risk-first after redeploy). Classification: universal (§11.4.17) — references no project-specific hardware; the consuming project supplies its reopens-count source per §11.4.35. Composes §11.4.5/§11.4.6/§11.4.55/§11.4.69/§11.4.107/§11.4.108/§11.4.115/§11.4.129/§11.4.130/§11.4.132/§11.4.135/§11.4.146. Propagation gate `CM-COVENANT-114-189-PROPAGATION` (literal `11.4.189`) + recommended gate `CM-MOST-REOPENED-EXTRA-DEPTH-LIVE-TEST` (the live-test suite orders + extra-depth-

scrutinises the highest-reopens-count cases FIRST, each with captured physical evidence on a clean deployment) + paired §1.1 mutation (drop the most-reopened extra-depth pass, OR run it AFTER the rest of the suite, OR PASS a most-reopened case on metadata-only evidence → the gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.189. Non-compliance is a release blocker. No escape hatch — no `--skip-most-reopened-depth`, `--reopened-same-as-any`, `--live-test-any-order`, `--metadata-pass-for-reopened-OK`, `--defer-reopened-scrutiny` flag.

§ . 2026 07 15 — Exact-reproduction-sequence mandate: when a working reproduction exists, the investigation MUST use ITS exact sequence — a deviating repro proves NOTHING (research-derived, - -)

Forensic anchor (FACT, 2026-07-15). An investigation concluded that a defensive "cover" mechanism "never engages" and that the defect it guards was therefore unfixed. The conclusion was WRONG: the hand-rolled reproduction drove the target with a DIFFERENT input than the existing test did (it pressed a HOME key where the test sent a media-pause key), so the hand-rolled path never reached the PRECONDITION the mechanism keys off. Re-driven with the TEST'S EXACT sequence, the mechanism provably ENGAGED. A deviating reproduction had produced a confident, captured-looking, and entirely false negative.

Every investigation / debugging pass / validation that reproduces a defect or exercises a code path MUST satisfy ALL of:

(1) Use the existing reproduction's EXACT sequence. When a working reproduction ALREADY EXISTS — a test, harness, script, or recorded sequence that demonstrably reaches the defect or the code path under investigation — the investigation MUST drive the target with THAT EXACT sequence: same inputs, same order, same timing, same preconditions, same environment / topology (§11.4.3). A hand-rolled approximation is NOT a substitute.

(2) A deviating variant MUST first PROVE it reaches the precondition. If a hand-rolled / simplified / alternative sequence IS used, it MUST FIRST be proven — with captured evidence — to reach the SAME precondition (the entry-state the mechanism or defect keys off) before ANY conclusion is drawn from it.

(3) A repro that misses the precondition proves NOTHING. It is NOT evidence of absence (§11.4.6 — absence of the observation is not observation of absence). Concluding "the defect is present" / "the defect is absent" / "the mechanism never engages" from a sequence that never reached the precondition is a §11.4 / §11.4.1 bluff at the INVESTIGATION layer — a FAIL-bluff when it wrongly condemns working code (and drives a needless "fix" into a healthy path), a PASS-bluff when it wrongly clears broken code.

(4) Record the exact driving sequence as captured evidence. Every reproduction MUST record the precise sequence it drove (commands / input events / order / timing) so it is replayable + auditable; any deviation from an existing reproduction MUST be explicit, justified, and precondition-proven.

STRENGTHENS §11.4.102 (systematic-debugging Phase 2 — reproduce), §11.4.146 (reproduce-first), and §11.4.115 (RED baseline on the broken artifact): those mandate THAT you reproduce; §11.4.199 mandates HOW — with the sequence PROVEN to reach the defect, not one that merely resembles it. Classification: universal (§11.4.17) — the consuming project supplies its concrete input / driving mechanism per §11.4.35. Composes §11.4.1 / §11.4.3 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.102 / §11.4.107 / §11.4.115 / §11.4.123 / §11.4.135 / §11.4.146. Propagation gate `CM-COVENANT-114-199-PROPAGATION` (literal `11.4.199`) + recommended gate `CM-EXACT-REPRO-SEQUENCE` (an investigation's conclusion cites the exact driving sequence used AND, where an existing reproduction exists, that the same sequence was used OR that the deviating sequence reached the precondition with captured proof) + paired §1.1 mutation (record a conclusion drawn from a deviating sequence with no precondition proof → the gate FAILs; strip the literal → the propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md` §11.4.199.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--approximate-repro-OK`, `--skip-precondition-proof`, `--hand-rolled-repro-suffices`, `--absence-proves-absence`, `--deviating-sequence-conclusion-OK` flag.

§ . . — Test-first (TDD) for ALL work, not only fixes + a minimum code-coverage floor that is NECESSARY-never-sufficient (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-07-22):

> "Any work we do MUST START by writing the test! We MUST FULLY comply and follow the TDD - The test driven development! It MUST BE applied for any change, implementation, wiring, tests, challenges and any other form of codebase we write! Bash scripts as well! Coverage with TDD MUST BE close to 100%, not less than 85% !!!"

The gap this closes. §11.4.43 mandates the test-first (RED → … → GREEN → VERIFY → DOCUMENT) workflow but its operative text binds **fixes** only ("Every fix MUST follow the 5-step TDD-fix workflow"); §11.4.115 refines that RED step onto the broken artifact; §11.4.146 binds reproduce-first. NONE of them binds a brand-new feature, a new wiring/integration, a new gate, a new Challenge, or a NEW BASH SCRIPT — none of which is a "fix", so none of which inherits a test-first obligation today. Separately, §11.4.27 and §11.4.169 mandate 100% **test-TYPE** coverage (breadth: WHICH KINDS of test exist per feature × platform) — an orthogonal axis that states no **code-coverage** figure anywhere. §11.4.224 closes exactly those two holes and claims nothing else as new.

(A) TEST-FIRST BINDS ALL WORK, not only fixes. For EVERY work product with an executable surface — a change, a new feature, an implementation, a wiring/integration, a refactor, a pre-build gate, a §1.1 mutation, a Challenge, test-harness code itself, AND **bash / shell scripts** — the test is WRITTEN FIRST, RUN FIRST, and OBSERVED TO FAIL for the right reason before the implementation exists; only then is the implementation written to make it pass. Where a broken artifact already exists the RED baseline is taken ON THAT ARTIFACT per

§11.4.115 (and its polarity switch converts the same source into the permanent §11.4.135 guard); where the work is genuinely new, the RED is the test failing against the absent/unimplemented behaviour, captured before the implementation lands. A test authored AFTER its implementation demonstrates only that the test AGREES with the code that already exists — never that it CATCHES the defect (the §11.4.43 PASS-bluff, here generalised to every work class). **What counts as "the test" per work class (§11.4.6 — named, never left to interpretation):** behaviour code → an executing test asserting the behaviour; a bash/shell script → an executing test that invokes the script through its REAL invocation path and asserts its real observable effect (exit status + emitted output + state delta), never a `bash -n` parse-check alone (parse-cleanliness is §11.4.67's separate obligation and proves nothing about behaviour); a pre-build GATE → its paired §1.1 mutation IS the test-first artifact, authored and observed to make the gate FAIL BEFORE the gate is trusted (a gate TRUSTED before its mutation was observed to FAIL is unvalidated instrumentation per §11.4.115(F) — the condition is observation-before-trust, NOT authorship order, so a gate and its mutation authored in the same commit are compliant when the mutation is observed to make the gate FAIL before the gate is relied upon); a Challenge → its scored assertion against captured evidence per §11.4.5/§11.4.69; a behaviour-bearing config/data change (an init-script line, a package-set manifest, an action-registry row — any committed DATA an executing consumer reads to decide behaviour) → an executing test that drives that CONSUMER through the changed value and asserts the changed observable behaviour (the clause-(F) executable-surface test reaches the config THROUGH its consumer — a config change is never parked under the prose-exempt class, which is strictly pure prose / governance / documentation).

(B) CODE-COVERAGE FLOOR — ≥ 85% MINIMUM, ~100% TARGET. Every governed codebase carries a measured code-coverage figure that MUST NOT fall below **85%**, with **as close to 100% as the domain permits** as the standing target (composing §11.4.169's "100% coverage as the domain permits" phrasing on its own test-type axis). The floor applies to the executable codebase INCLUDING bash/shell scripts — a shell corpus exempted from measurement is exactly the gap the operator's "Bash scripts as well!" closes.

(B.1) SEVEN-TYPE BREADTH — the 100% AIM across a canonical test-type enumeration (extension, 2026-08-15). The 85% floor is a LINE-COVERAGE proxy across the executable codebase; the 100% AIM is the STANDING TARGET across the seven-canonical-test-type × executable-surface product per operator

mandate 2026-08-15. The seven canonical types every consuming project MUST cover (where applicable) are: (1) UNIT, (2) INTEGRATION, (3) END-TO-END (E2E), (4) FULL AUTOMATION, (5) SECURITY, (6) PERFORMANCE AND BENCHMARK, (7) ANTI-BLUFF — each carrying its own §11.4.27 mandatory-breadth obligation (which this anchor cross-references, and §11.4.27's clause (B) is amended in step). A project measuring 100% line coverage on UNIT tests alone while omitting one of the seven applicable types is a §11.4.224(B.1) violation regardless of the line-coverage number (§11.4.201(8) metric-validated-against-done: the correct end-state is line coverage AT target ACROSS all seven applicable types, never on one type alone). Honest boundary §11.4.6 / §11.4.17: a test type genuinely inapplicable to a project (e.g. a library with no runtime surface has no E2E user-journey to drive) is HONESTLY marked as §11.4.3 SKIP-with-reason at the type-breadth axis, never silently omitted; the universal DISCIPLINE is unchanged, the applicability of each type is the consumer's §11.4.35 DATA. `CM-COVERAGE-FLOOR` is EXTENDED — the measured numerator STILL counts only RED-capable tests, PLUS the gate FAILs on missing test-type-breadth coverage per the seven-type enumeration (a project measuring 90% line coverage but with zero ANTI-BLUFF tests → gate FAILs); a paired §1.1 mutation that strips one enumerated type from the type-breadth check → the gate FAILs.

(C) COVERAGE IS NECESSARY, NEVER SUFFICIENT — the anti-gaming clause (§11.4.201(8) / §11.4/§1.1). A coverage percentage is trivially gameable and MUST NEVER be reported as evidence a change is correct: **MEASURED FACT (in-session, 2026-07-22, this host):** a line-granular coverage instrument reported an `if …; then …; else …; fi` line as COVERED while the `else` branch was NEVER taken — the metric read fully-covered on a line half of whose behaviour was untested; and a test that EXECUTES a line while ASSERTING NOTHING raises the same number by the same amount as a test that proves the behaviour. Therefore: (1) the REAL bar remains the §11.4 anti-bluff covenant + §1.1 — every test MUST catch its own negation, PROVEN by a paired mutation that makes it FAIL; a line executed by an assertion-free test is NOT covered in the sense that matters; (2) the floor is a MINIMUM ON A MEASURABLE PROXY, never a certificate of correctness — citing "≥ 85% covered" as proof a change works is a §11.4 PASS-bluff at the metric layer; (3) the floor NEVER stands alone — it composes with the §1.1 mutation pair, the §11.4.5/§11.4.69/§11.4.107 captured-evidence bar, and the §11.4.108 runtime signature, and it does NOT displace any of them.

(D) METRIC VALIDATION AGAINST THE DEFINITION OF DONE (§11.4.201(8)) — split verdict, stated honestly. §11.4.201(8) requires every gating metric to be evaluated at the correct end-state and REFUSED if the correct end-state does not move it to target. **Directional validation, performed 2026-07-22:** at the correct end-state — every behaviour test-driven, every test catching its own negation — the lines implementing that behaviour ARE executed by their own tests, so the metric DOES move to target; the correct end-state does NOT refute the floor, and the floor is therefore VALID as a NECESSARY condition. **The converse is REFUTED** by the clause-(C) measured gameability fact: reaching the floor does NOT imply the end-state — hence necessary-never-sufficient, and hence clause (C) is load-bearing rather than decorative. **OWED, not claimed (§11.4.197):** the EMPIRICAL calibration of the specific 85% threshold against a given project's REAL corpus — whether a genuinely correct end-state on THAT corpus lands at 85%, higher, or lower — has NOT been performed and MUST be tracked as a §11.4.197 work item by every consuming project that enforces the floor; this anchor asserts no project has already filed it (asserting an unverified tracker state would be the §11.4.6 violation this clause exists to prevent). **Interim precedence** (mirroring the precedent §11.4.201(8) itself set for the §11.4.135 ratchet): until that calibration lands the ≥ 85% floor KEEPS GATING — an uncalibrated floor blocking a merge is bounded, visible and operator-resolvable, whereas disarming the only mechanism that surfaces untested code silently re-opens the hole (the §11.4.101 reversible-safe choice); the calibration is OWED WORK, never a licence to switch the floor off.

(E) HOW COVERAGE IS MEASURED — per language, honestly (§11.4.6 / §11.4.35). The consuming project supplies its per-language coverage instrument as DATA per §11.4.35; the UNIVERSAL rule is that an instrument MUST exist and MUST actually be RUN — **a floor nobody can measure is an unmeasurable gate and is itself a §11.4.201 bluff**, and a percentage nobody measured is a §11.4.6 fabrication. **Measured host reality (2026-07-22, the project instantiating this anchor):** Go coverage is available built-in (`go test -coverprofile` + `go tool cover`); C/C++ via `gcov` ; and **for bash/shell NO dedicated coverage tool was installed** — `kcov` , `bashcov` , `bats` , `shellspec` and `shunit2` were ALL probed ABSENT (`shellcheck` IS present but is a LINTER, not a coverage instrument, and satisfies nothing here). **PROVEN-FEASIBLE bash mechanism (measured in-session, never asserted):** trace-based line accounting needs NO additional tooling — `PS4='+COV:${BASH_SOURCE##*/}:${LINENO}:' bash -x <script>` emits the executed line numbers, and executed-lines ÷ executable (non-

blank, non-comment) source-lines yields line coverage; a probe run correctly identified an uncovered function body as uncovered, and subshell bodies ARE traced. **HONEST LIMITS of that mechanism (measured, stated as fact):** it is LINE coverage, NOT branch coverage — the clause-(C) same-line `if/else` counted as covered with one branch never taken; `set +x` regions and traps are not accounted. It is therefore a FLOOR-measuring instrument only; where branch-level assurance is required the project MUST install a real coverage tool (e.g. `kcov`) or record the shortfall honestly. **Where NO instrument exists** for a language on a host, the honest responses are to INSTALL one, or to record an §11.4.3/§11.4.69 SKIP-with-reason plus a tracked §11.4.197 item — NEVER an invented percentage, and NEVER a silently-dropped floor. **EXCLUSION-LIST FENCE (amendment, remediation round of 2026-07-22 — the §11.4.135 exemption pattern applied to the coverage corpus).** The corpus-scope exclusion list is NOT an unconstrained consumer knob: an exclusion is legal ONLY via a CHECKED-IN exclusion list, with EVERY entry justified from a CLOSED class set — `generated-code` (output of a committed generator per §11.4.77), `vendored-third-party` (executable code the project does not author), `non-shipping-fixtures-and-golden-assets` (test fixtures / golden files that never execute in the shipped product) — and enumerated as honest gaps in the coverage report, never silent (the exact discipline §11.4.135 applies to topology exemptions: legal ONLY via a checked-in map, enumerated as honest gaps — never silent). Excluding FIRST-PARTY executable code additionally REQUIRES a tracked §11.4.197 work item naming the plan to bring it into scope. An unlisted, unjustified, or un-tracked-first-party exclusion VOIDS the measured figure — a consumer that excludes most of its first-party executable code and truthfully measures ≥ 85% on the remainder has satisfied the operator's floor in letter and voided it in substance; that silent-evasion channel is exactly what this fence closes.

(F) HONEST BOUNDARY / SCOPE (§11.4.6). "All work" means work products with an EXECUTABLE surface. A pure prose / governance / documentation edit has no executable surface and is NOT coverage-scoped — its discipline is §11.4.44 / §11.4.65 / §11.4.106 / §11.4.212 plus the §11.4.142 universal review; quoting a coverage percentage for a prose document would be meaningless, and demanding one would be a §11.4.201(1) false-positive refusal. BUT a governance change that SHIPS an executable artifact (a gate, a hook, a script, a mutation) is FULLY in scope for both (A) and (B). Test-first does NOT replace the §11.4.108 four-layer verification, the §11.4.40 full-suite retest, or the §11.4.185 manual-QA final gate — it

PRECEDES them. And §11.4.224's CODE-coverage axis is DISTINCT from §11.4.27 / §11.4.169's 100% TEST-TYPE coverage (breadth of test kinds per feature × platform): both apply simultaneously, neither substitutes for the other, and satisfying one is never evidence for the other.

Classification: universal (§11.4.17) — references no project-specific hardware, vendor, or language; the consuming project supplies its per-language coverage instruments, its corpus scope, its CHECKED-IN clause-(E)-fenced exclusion list, and its threshold calibration per §11.4.35. Composes §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.27 / §11.4.35 / §11.4.43 (generalises its fix-scoped test-first to ALL work) / §11.4.50 / §11.4.66 (brownfield adoption is an operator question) / §11.4.67 / §11.4.69 / §11.4.77 / §11.4.85 / §11.4.107 / §11.4.108 / §11.4.110 / §11.4.115 / §11.4.122 / §11.4.135 / §11.4.142 / §11.4.146 / §11.4.169 (orthogonal test-TYPE axis) / §11.4.185 / §11.4.197 / §11.4.201(8) / §11.4.202 / §1.1. Propagation gate `CM-COVENANT-114-224-PROPAGATION` (literal `11.4.224`) + recommended gates `CM-TEST-FIRST-ALL-WORK` (every commit adding an executable artifact carries its test, and that test's failing-first observation is recorded — a new script/gate/feature landing with no test, or with a test whose RED was never captured, FAILs) + `CM-COVERAGE-FLOOR` (a measured coverage figure exists per declared language, was produced by a real instrument run rather than asserted, and is ≥ the declared floor; a declared language with NO instrument FAILs rather than silently passing; **the measured numerator counts ONLY lines covered by RED-CAPABLE tests** — every test contributing to the measured corpus MUST carry a recorded failing-first run (its clause-(A) RED record) OR a paired mutation observed to make it FAIL, with sampling-based mutation verification an acceptable stated implementation and a contributing test carrying ZERO assertions rejected by construction, so assertion-free execution cannot raise the measured figure and ONE real RED-captured test plus assertion-free padding to 85% FAILs this gate; AND the gate FAILs on any corpus exclusion that is absent from the checked-in list, not justified from the clause-(E) closed class set, or first-party without its tracked §11.4.197 item) + paired §1.1 mutations (land an executable artifact with no test → `CM-TEST-FIRST-ALL-WORK` FAILs; record a coverage percentage with no instrument run behind it, or drop a language's measurement entirely, or pad the numerator with an assertion-free test, or slip an unjustified exclusion into the corpus scope → `CM-COVERAGE-FLOOR` FAILs; strip the literal → the propagation gate FAILs). **Brownfield adoption is DELIBERATELY UNDEFINED by this anchor (§11.4.66):** when `CM-COVERAGE-FLOOR` first lands on a project whose legacy corpus measures below 85%, the

gate-code work item MUST surface the adoption question to the OPERATOR per §11.4.66 BEFORE first enforcement — candidate options to present: immediate hard floor (day-one RED until the corpus is raised) / a ONE-TIME monotone-decrease ratchet per the §11.4.135 precedent / per-corpus phase-in (per language or module as each enters scope) / floor-on-changed-code-only with a scheduled full-corpus deadline — recording the operator's answer as consumer DATA (§11.4.35); this anchor does NOT autonomously weaken the operator's unconditional "not less than 85% !!!" with an invented ratchet (a §11.4.122-class decision the operator owns). Gate-code = a separate work item — the CONTRACT is landed here, the gate CODE is NOT claimed shipped (§11.4.6).

Canonical authority: constitution submodule `Constitution.md` §11.4.224.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--test-after-OK`, `--skip-red-first`, `--bash-exempt-from-tests`, `--coverage-not-applicable`, `--below-floor-OK`, `--coverage-proves-correct` (11.4.238), `--unmeasured-percentage-OK`, `--assertion-free-test-counts` flag exists.

§ . . (2026-08-08) — Automated QA must be the DISCOVERER: manual QA finds nothing new, and anything it finds is a coverage escape (User mandate, - -)

Forensic anchor — verbatim user mandate (2026-08-08): *"Helix QA full websites testing MUST find all such issues, not us! Once we do manual QA we shall not be able to find anything! THIS IS MANDATORY RULE / CONSTRAIN / GOAL to achieve with this and any other project and MUST BE part of the root constitution!"*

Forensic case study (FACT, 2026-08-08). In a single cycle on a governed project, five real defects were found — a health endpoint that returned `{"status":"healthy","components":[]}` and was structurally incapable of ever reporting unhealthy; a gateway routing every completion at a port nothing served, so the product's primary capability returned HTTP 500 for an indeterminate period; two submodule pins recorded on `main` that upstream answers with `not our ref`, so no fresh clone could check them out; and two hardware-profiler defects reading a key from a file where it does not exist. **Every one was found by an agent reading source code or probing by hand. Not one was found by the**

automated QA system, which reported green throughout. The defects were real, user-visible, and long-lived; the QA suite's silence about them was the actual defect.

The mandate (ALL hold):

(A) THE AUTOMATED QA SYSTEM IS THE DISCOVERER, NOT THE CONFIRMER. The project's automated QA system (HelixQA per §11.4.27, or the consuming project's equivalent full-system/full-website test regime per §11.4.35) MUST be the mechanism that FINDS defects. Discovery by a human, by an operator's manual pass, by an agent reading code, or by an end-user report is a **failure of the QA system**, not a success of the finder. The finding is still valuable and still gets fixed — but the QA gap it exposes is a defect of equal standing, and closing only the reported defect while leaving the gap is the violation this anchor forbids.

(B) MANUAL QA MUST FIND NOTHING NEW. By the time manual QA runs (§11.4.185), the automated regime MUST already have surfaced everything it would find, so manual QA functions as a CONFIRMATION and sufficiency gate — never as the discovery mechanism. The target state is explicit and measurable: *a manual QA pass that discovers zero previously-unknown defects.* A manual pass that keeps finding new defects is evidence the automated regime is not yet doing its job, regardless of how green it reports.

(C) EVERY ESCAPE TRIGGERS THE §11.4.138 LOOP, WIDENED. When ANY defect is discovered outside the automated QA system — manual QA, operator observation, end-user report, agent code-reading, or incidental discovery during unrelated work — it triggers, before the fix may close: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause the defect itself; (2) a **coverage-escape audit** identifying WHY the automated regime did not find it, cited to the specific missing check, unexercised path, absent assertion, or untested surface — never "we didn't think of it"; (3) a **new or strengthened automated check** that WOULD have found it, registered into the standing regime (§11.4.135) with its §11.4.115 RED capturing the escaped defect; (4) the audit committed as captured evidence. §11.4.138 already mandates this for operator-found defects that a green suite passed; §11.4.238 widens the trigger to EVERY out-of-band discovery channel, because the failure is identical whoever happens to notice first.

(D) COVERAGE IS MEASURED BY WHAT THE REGIME CAN CATCH, NOT BY WHAT IT RUNS. A QA regime's worth is its demonstrated ability to FAIL on a real defect, not its pass count, test count, or run duration. Every automated surface MUST be falsifiable per §1.1 — a check whose paired mutation does not make it fail is decoration, and a suite composed of such checks will report green through exactly the defects this anchor exists to prevent. The regime MUST exercise real user-reachable paths end-to-end (§11.4.98 no manual intervention, §11.4.143 real user journeys, §11.4.136 real content, §11.4.158/§11.4.159/§11.4.160 recorded + read-the-screen verification), because a check that never traverses the path a user traverses cannot find what a user finds.

(E) THE GOAL IS DIRECTIONAL AND MEASURED. Projects MUST track the discovery-channel split — how many defects in a cycle were found by the automated regime versus out-of-band — and drive the out-of-band share toward zero over successive cycles. Honest boundary (§11.4.6): this anchor does NOT claim a QA regime can ever be provably complete (§11.4.118 — absence of evidence is not evidence of absence), and it does NOT license closing an escape by weakening the definition of "defect". It mandates that every escape measurably strengthens the regime, so the channel that finds defects shifts from people to machines cycle over cycle.

Composes §11.4.27 (the QA system) / §11.4.138 (operator-escape bluff-audit — §11.4.238 widens its trigger set) / §11.4.185 (manual QA as final sufficiency gate — §11.4.238 makes it a confirmation, never a discovery, step) / §11.4.118 (enumerated discovery pressure) / §11.4.135 (standing regression guards) / §11.4.115 / §11.4.98 / §11.4.136 / §11.4.143 / §11.4.158 / §11.4.159 / §11.4.160 / §11.4.102 / §11.4.6 / §1.1.

Classification: universal (§11.4.17) — a platform-neutral QA-ownership discipline reusable by ANY project with an automated QA regime and a manual QA step; the consuming project supplies its concrete QA system, its surface inventory, and its discovery-channel ledger per §11.4.35. Propagation gate `CM-COVENANT-114-238-PROPAGATION` (literal `11.4.238`) + recommended gates `CM-QA-IS-THE-DISCOVERER` (every defect closed in a cycle records its discovery channel; an out-of-band channel requires a committed coverage-escape audit + a registered automated check that would have caught it) and `CM-MANUAL-QA-FINDS-NOTHING-NEW` (a manual QA pass reporting new defects blocks the release tag until each has its coverage-escape audit and new check) + paired §1.1 meta-test mutation

(strip the literal → propagation gate FAILs; close an out-of-band-discovered defect with no coverage-escape audit → `CM-QA-IS-THE-DISCOVERER` FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

EXTENSION — Discovery-channel record schema + escape-ratchet no-data-point rule (spec-derived, speckit 002-anti-slop-enforcement, 2026-08-26). §11.4.238(C) already states the CONCEPT — a coverage-escape audit citing the specific missing check that would have caught an out-of-band-discovered defect — but does not fix the record's field schema, and §11.4.238(E) states only that the out-of-band share is driven "toward zero over successive cycles," a directional framing this project's own spec.md identifies as amended for the same reason SC-003's original wording was: *"'trend to zero over successive cycles' names neither an input condition nor an observable pass/fail outcome, so it was not machine-checkable ... the ratchet is the decidable form"* (spec.md:220, SC-008 amendment note). This extension supplies the decidable schema and ratchet. **(1) Discovery-channel record schema.** Per spec.md:176 (FR-043): every recorded defect MUST carry a discovery channel drawn from the CLOSED SET `{automated_seam, manual_qa, operator, end_user, agent_inspection}` (or the consuming project's declared equivalent per §11.4.35) AND a `should_have_been_caught_by` value naming a specific seam or gate, OR the literal `none` with a WRITTEN justification; a record missing either field is REFUSED. The record MUST be WRITTEN BY THE VERIFIER SEAM, never by the producer of the defect's fix — a producer-authored discovery-channel record is refused (the same producer≠verifier separation §11.4.240 already mandates, applied to the record OF the escape, not only the record of the fix). **(2) Escape count is monotone-decreasing from a recorded baseline, and never increases.** Per spec.md:177 (FR-044): a cycle whose escape count exceeds the recorded baseline is REFUSED at the release seam; a cycle whose escape count is lower moves the baseline down. **(3) A manual-QA-not-run cycle contributes NO DATA POINT — it MUST NOT lower the baseline.** Per spec.md:177 (FR-044, verbatim): *"a cycle in which the manual pass did not run, that cycle MUST contribute no data point and MUST NOT lower the baseline"* — a zero escape count from a cycle nobody inspected is not a measurement (the §11.4.201(6) false-null pattern applied to this specific ratchet: an unmeasured cycle and a genuinely-clean cycle both read as zero, and only the `manual_qa_ran` flag distinguishes them). **(4) Records tagged `should_have_been_caught_by: none` are counted and reported SEPARATELY.** Per spec.md:220: *"so narrowing the definition of an escape is visible rather than silent"* — a project cannot lower its

escape ratchet by quietly reclassifying escapes as uncatchable-by-design; every such reclassification is a visible, separately-tallied line item, never absorbed into the main count. **Honest boundary (§11.4.6), stated in the same spec.md amendment note (SC-008) this extension grounds itself in:** *"Validated against the definition of done: at the correct end-state, every escape caught by a seam, the count reaches 0, so the ratchet is a valid necessary condition. It is not sufficient — zero also results from nobody looking, which is what the no-data-point rule and the separate* `none` *tally exist to expose."* Composes §11.4.201(6)-(8) (the false-null + metric-validated-against-definition-of-done pattern this ratchet instantiates) / §11.4.135(5) (the release-verdict-coverage monotone-decrease adoption ratchet — this extension's escape-count ratchet is the SAME decidable-monotone-baseline pattern applied to §11.4.238's discovery-channel domain, not a novel mechanism) / §11.4.261 (the standing zero-shortcomings monotone-decreasing findings ledger — the general-purpose instantiation of the identical never-increases/may-decrease/no-silent-narrowing discipline this extension's escape ratchet and `none`-tally separate-reporting rule both specialise). Recommended mechanism gates `CM-DISCOVERY-CHANNEL-RECORD-COMPLETE` (every recorded defect carries a closed-set channel + a `should_have_been_caught_by` seam-name or a justified `none`; the record is verifier-seam-written, never producer-authored) + `CM-ESCAPE-RATCHET-NO-DATA-POINT-HONEST` (a cycle whose escape count exceeds the recorded baseline refuses the release seam; a cycle with `manual_qa_ran` false contributes no data point and does not lower the baseline; `none`-tagged records are tallied separately from the main escape count) + paired §1.1 mutations (submit a defect record with no channel, or a `should_have_been_caught_by: none` with no justification → the schema gate MUST FAIL; author a discovery-channel record from the producer of the defect's own fix → the schema gate MUST FAIL; run a cycle with `manual_qa_ran` false and zero recorded escapes, then assert the baseline lowers → the ratchet gate MUST FAIL; golden-FALSE per §11.4.201(1): a genuine manual-QA cycle with a real zero-escape result → the ratchet gate MUST allow the baseline to lower). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.238.

Non-compliance is a release blocker regardless of context. No escape hatch — no `--manual-qa-may-discover`, `--skip-coverage-escape-audit`, `--found-by-human-is-fine`, `--close-escape-without-new-check`, `--qa-green-is-enough`; `--discovery-channel-not-tracked` flag exists.

---

§ . . – Critical-invariant work-class Definition of Done mandate – money/safety/availability/integrity code MUST include failure-path scenarios as gates, not merely happy-path passes (research-derived, - -)

Compact summary: every project's Definition of Done (§11.4.15/§11.4.16 lifecycle + §11.4.33 closure vocabulary) MUST identify a closed CRITICAL-INVARIANT WORK CLASS — code paths where a single incorrect outcome is unrecoverable in the window between failure and remediation (money movement, safety-of-life or safety-of-property outputs, availability of the primary user-facing capability, integrity of the persistent record) — and MUST require that a change touching this class carry, as part of its acceptance-criteria set, EXPLICIT FAILURE-PATH SCENARIOS turned into passing tests BEFORE closure. A DoD that stops at "code compiles + happy-path test is green" for a critical-invariant change is a machine for generating reopened tickets on exactly the paths that can least afford them (extraction file line 49 — **CITATION NOTE (2026-08-20 citation audit, §11.4.6): the "extraction file" offsets cited throughout §11.4.239-§11.4.254 are absolute offsets into a consolidated extraction artifact that is NOT tracked in this repository and whose numbering does NOT resolve against the tracked corpus at `docs/research/ai_curriculum_2026_08/source_corpus/` — the highest cited offset (6490) exceeds the 5838-line total of ALL NINE module files concatenated, and every module-scoped offset cited below either exceeds its module's own line extent or points at unrelated text. Each extraction offset is therefore retained ONLY as an un-resolvable provenance pointer, NEVER as a resolvable location, and is accompanied by a VERIFIED `module_NN:LINE` locator measured against the tracked corpus with a control-needle-proven instrument per §11.4.201(7)(b). The corpus itself landed 2026-08-19, AFTER these anchors landed, so it corroborates them; it is not their derivation**

source. — module 27, verified locator `module_27:30` : "A Definition of Done that stops at 'the code compiles and the happy-path test is green' is a machine for generating reopened tickets on the paths that move money"; line 258 [verified locator `module_27:231`]: the load-bearing DoD lines are the failure-mode tests and captured runtime evidence). §11.4.239 does NOT invent a new lifecycle or a new evidence class; it PROMOTES the failure-path coverage the constitution's anti-bluff family (§11.4/§11.4.5/§11.4.69/§11.4.107) already demands into a MECHANICAL PRECONDITION of any closure-status write on the critical-invariant class, so the "happy-path green means done" bluff cannot ship on money/safety/availability/integrity code by construction.

(A) THE CRITICAL-INVARIANT WORK CLASS — CLOSED SET, CONSUMER-BOUND. Every consuming project MUST declare, as DATA per §11.4.35, its own concrete instantiation of the four critical-invariant categories. The universal categories are: (i) MONEY — any code that credits, debits, transfers, holds, releases, refunds, or reconciles a monetary value or its digital equivalent; extraction line 49 [verified locator `module_27:30`] anchors the canonical example ("a donation either completed correctly or it did not; there is no partial credit and no maintenance window"); (ii) SAFETY — any code whose outputs actuate physical hardware, medical devices, transport systems, industrial controls, or otherwise reach into the physical world where a wrong output can cause harm; (iii) AVAILABILITY — any code on the critical path of the product's primary user-facing capability whose failure degrades the user's ability to use the product at all (login, primary action, data read on the load-bearing path); (iv) INTEGRITY — any code that mutates the persistent record whose corruption cannot be silently repaired (append-only ledgers, audit logs, cryptographic anchors, source-of-truth writes governed by §11.4.93/§11.4.95). A code region is IN the class when at least one category applies; a region OUT of the class stays governed by the ordinary DoD. §11.4.239 does NOT claim the four categories are exhaustive for every domain — the consuming project supplies extensions (regulatory-compliance outputs, privacy-critical PII flows, legal-hold data) per §11.4.35, and each extension carries the same failure-path-mandatory discipline.

(B) FAILURE-PATH SCENARIOS AS FIRST-CLASS ACCEPTANCE CRITERIA — NOT AFTERTHOUGHTS. For every change touching the critical-invariant class per (A), the change's acceptance-criteria set MUST enumerate the domain-specific failure paths BEFORE the code is authored, each rendered as a Given/When/Then scenario (module 27 acceptance-criteria pattern — module attribution VERIFIED

CORRECT 2026-08-20: `Given/When/Then` occurs in module 27 ONLY across the whole nine-module corpus, verified locators `module_27:94`, `module_27:268`, `module_27:275`, `module_27:318`; extraction lines 395-428 canonical example, offsets un-resolvable per the §11.4.239 CITATION NOTE) whose "Then" observes a real user-observable outcome (a ledger row, a settled charge, a state delta) rather than "no error" (§11.4/§11.4.69). The canonical failure-path scenario set for MONEY code (extraction lines 400-428; verified locator `module_27:312`) is the reference minimum: (1) FRESH — the happy path; (2) IDEMPOTENT RETRY — same key + same payload produces no new side effect; (3) SAME-KEY-DIFFERENT-PAYLOAD — the request is rejected, not quietly served (extraction line 430 [verified locator `module_27:312`]: "the two criteria juniors omit are same-key-different-payload ... and the crash-in-the-middle"); (4) CRASH-IN-THE-MIDDLE — the process is killed between the external commit and the local persist and re-driven, converging to exactly-once. Analogous minimum scenario sets for SAFETY (nominal + fault-injected upstream + degraded actuator + power-loss mid-actuation), AVAILABILITY (nominal + dependency degraded + dependency down + retry storm), and INTEGRITY (nominal write + concurrent conflicting write + mid-write crash + corruption-detected-on-read) are consumer-supplied per §11.4.35 with the SAME "failure paths as gates, not as afterthoughts" property. Every scenario becomes a passing test BEFORE closure; a scenario omitted is a §11.4.239 finding of §11.4/§11.4.1 severity at the acceptance-criteria layer, NOT a "we'll add it if it bites us" deferral.

(C) MECHANICAL PRECONDITION AT THE STATUS-WRITE SEAM. The failure-path-coverage precondition binds through the existing §11.4.146(D3) status-custody chain, not a parallel mechanism: a done/ready/closed-status write against an item whose implementing diff touches the (A)-declared critical-invariant class is REFUSED unless the item's registry row cites acceptance-criteria coverage of the domain's declared failure-path scenario set (with each scenario mapped to a §11.4.115(F) machine-written verdict pair for the tests that exercise it, and each verdict carrying its class-matched evidence per §11.4.226). The refusal is at the SEAM (the workable-items engine per §11.4.93 refuses the write), not a prose review guideline, per §11.4.205's "seams not prose" pattern. An item whose scenario-map lists a scenario as "N/A" MUST cite the §11.4.6-honest reason the scenario does not apply (SAFETY change with no upstream fault possible on this actuator; a MONEY read-only endpoint with no crash-in-the-middle exposure) — a bare "N/A" is treated as omission.

(D) COMPOSITION WITH §11.4.108 / §11.4.129 / §11.4.185 / §11.4.238. §11.4.108's four-layer verification (SOURCE → ARTIFACT → RUNTIME → USER-VISIBLE) remains the layer discipline for every failure-path scenario — the acceptance test does NOT close on source-green, it closes on runtime-class evidence per §11.4.226. §11.4.129's huge-blocker STOP-fix-all-full-RESTART protocol governs the response WHEN a failure-path scenario surfaces a defect during release validation. §11.4.185's manual-QA sufficiency gate is un-weakened — automated failure-path coverage is NECESSARY, never sufficient, and the operator's final human oracle remains the terminal check. §11.4.238's automated-QA-as-discoverer mandate binds the automated regime to FIND the failure-path defects BEFORE manual QA — a manual-QA session that discovers a critical-invariant failure-path defect the automated regime missed is a §11.4.238 coverage-escape audit AND a §11.4.239 acceptance-criteria audit (the scenario should have been enumerated at authoring time).

(E) HONEST BOUNDARY (§11.4.6). Failure-path coverage is DIRECTIONAL — it forbids the "happy path suffices" shape and mandates a domain-appropriate minimum failure-path enumeration; it does NOT claim exhaustive enumeration of every possible failure mode (that direction crosses into formal-methods and §11.4.118 discovery-pressure territory). The domain-specific minimum scenario sets in (B) are the STARTING POINT, not the ceiling — extraction line 210 [verified locator `module_27:211`] documents the ratchet: "the fix is never 'try harder'; it is to add the specific missing gate ... so the next occurrence is caught before close," so every escape becomes a permanent addition to the scenario set (§11.4.135 regression-guard family applied to the acceptance-criteria set). Composes §11.4.15 (status lifecycle) / §11.4.16 (type set) / §11.4.33 (closure vocabulary) / §11.4.35 (consumer-owned category extensions) / §11.4.43 (TDD-fix — the failure-path test is the RED before the fix) / §11.4.69 (positive-evidence taxonomy — every "Then" is an observed effect) / §11.4.108 (four-layer verification stays in force per scenario) / §11.4.115(F) (machine-written verdict pairs bind each scenario) / §11.4.135 (each failure-path scenario becomes a permanent regression guard) / §11.4.146(D3) (status-custody chain enforces the precondition) / §11.4.185 (manual QA remains the sufficiency gate) / §11.4.205 (seam-not-prose enforcement) / §11.4.226 (evidence-class-at-closure ranks the scenario's captured evidence) / §11.4.238 (automated QA discovers, manual QA confirms). Extraction sources [offsets un-resolvable per the §11.4.239 CITATION NOTE; BOTH module attributions AUDITED 2026-08-20 and VERIFIED CORRECT]: module 27 ("Definition of Done,

Acceptance Criteria & Workable-Items Governance") lines 49-53, 245-258, 336-365, 395-430, 748-752 — verified locators `module_27:30`, `module_27:211`, `module_27:231`, `module_27:312`, plus the Given/When/Then acceptance-criteria pattern at `module_27:94` / `268` / `275` / `318` (offsets 748-752 additionally exceed module_27's own 668-line extent, which is itself the proof the offsets are extraction-file-absolute rather than module-relative); module 28 ("Anti-Bluff Verification & Release Validation") lines 1084-1142 verification-vs-validation depth — verified locators `module_28:139`, `module_28:141` (offsets 1084-1142 likewise exceed module_28's own 640-line extent).

Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-239-PROPAGATION` (literal `11.4.239`) + recommended gate `CM-CRITICAL-INVARIANT-FAILURE-PATH-DOD` (a change whose implementing diff touches the consumer-declared critical-invariant class per (A) carries an acceptance-criteria map naming each domain-required failure-path scenario per (B); a scenario listed "N/A" carries a §11.4.6-honest reason; a closure-status write against an item whose map is incomplete or whose failure-path tests lack §11.4.115(F) machine-written verdicts is refused at the status-write seam per (C)) + paired §1.1 mutation (submit a MONEY-touching change whose acceptance-criteria set enumerates only the happy path → gate FAILs; strip the same-key-different-payload scenario from an idempotent-endpoint change → gate FAILs; author a scenario whose "Then" observes only "no error" rather than a state delta → the composed §11.4.69 gate FAILs on evidence class; strip the literal → propagation FAILs; gate-code = separate work item, NOT claimed shipped §11.4.6/§11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.239. Non-compliance is a release blocker. No escape hatch — no `--happy-path-suffices-for-money`, `--skip-failure-scenarios`, `--defer-failure-path-tests`, `--criticality-not-applicable-without-reason`, `--n/a-without-justification`, `--closure-without-scenario-map` flag.

---

§ . . — Bisection-not-blame regression cause-finding + failure modes (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.242. When a previously-working feature/behaviour regresses, the FIRST-choice localisation mechanism is BISECTION (`git bisect` or equivalent monotone-search, O(log N)) over the ordered commit range between the last known-good tag per §11.4.114 and current HEAD, NOT blame-log inspection, NOT speculative root-cause hunting. MONOTONE-SEARCH INVARIANT: bisection valid only when observable is monotone across the range (broken stays broken); non-monotone (flaky, environment-dependent, timing-race) MUST stabilise first — bisection over flaky signal produces a WRONG "guilty" commit with the same confidence as a correct one (§11.4/§11.4.1 bluff at the localisation layer). FAILURE-MODE CATALOGUE (closed 6-class set): (1) flaky/non-monotone observable, (2) build-broken commits mistaken for "bad" (use `bisect skip` exit 125), (3) merge-commit introductions (defect in conflict resolution not either parent), (4) external/environmental changes, (5) delayed-fuse introductions (latent commit + later trigger), (6) test-infra regressions masquerading as product regressions. `git bisect run` script MUST use exit-code discipline (0=good, 1-124=bad, 125=skip, 128+=abort); mapping every non-zero to "bad" collapses the taxonomy. Bisection RESULT is captured evidence per §11.4.5/§11.4.69 feeding §11.4.102 systematic-debugging's root-cause phase. Honest boundary §11.4.6 — bisection localises the INTRODUCING COMMIT of a MONOTONE regression, does NOT prove that commit's change is the ROOT CAUSE. Composes §11.4.5 / §11.4.6 / §11.4.69 / §11.4.102 / §11.4.114 / §11.4.115 / §11.4.201 / §11.4.224 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-242-PROPAGATION` (literal `11.4.242`) + recommended gate `CM-BISECTION-BEFORE-BLAME` + paired §1.1 mutation.

EXTENSION — DELTA DEBUGGING: bisection is not only over commits (2026-08-20). §11.4.242's subject is regression LOCALISATION, and its honest boundary already states its own limit — it localises *"the INTRODUCING COMMIT of a MONOTONE regression"*. That limit leaves two whole classes unaddressed: a defect with no known-good commit (present since inception, or input-dependent), and a defect whose discriminating variable is not a commit at all. The AI-curriculum corpus (module 33) supplies the generalisation and states the containment relation in the direction that matters: **`git bisect` is delta debugging over commits**, not the reverse. Delta debugging (Zeller; the corpus does not use the name `ddmin`

and neither does this clause) is defined as: *"given a large failing input, it systematically removes chunks and re-tests, keeping the input failing while shrinking it, until removing any remaining piece makes the failure disappear — a 1-minimal failing case. The algorithm starts by trying to remove large chunks and progressively removes smaller ones, so it converges quickly, roughly binary-search-like on the input."* The corpus's enumerated generalisation axes, verbatim: *"Bisect a config — remove half the flags/settings and see if the bug survives. Bisect the data — which record in the batch triggers it? Bisect the change set — git bisect is delta debugging over commits. Bisect dependencies — which upgraded package flipped behavior?"* (feature flags are named as a further axis in the same module; STATE is NOT an enumerated axis and is not claimed as one here — §11.4.6). **The mandate.** When a failure is not a monotone commit-range regression, or when a commit-bisection returns a change too large to explain the failure, the investigation MUST narrow along the discriminating axis — input, config, data, dependency, or flag — to a MINIMAL failing case before a fix is designed, and the minimal case becomes the §11.4.115 RED test's input. **The predicate carries the same monotone-search invariant §11.4.242 already imposes:** the corpus requires *"a stable, fast, unambiguous test predicate — 'does this input still trigger this failure' (not a different one)"*, and for a non-deterministic failure warns that *"minimization is noisy because a 'good' reduction step might just have gotten lucky, so I run the predicate multiple times per step or accept a probabilistic minimum"* (composes §11.4.50 deterministic consistency and §11.4.248 — stabilise or bound the noise before trusting the reduction). Honest boundary (§11.4.6): minimisation produces the SMALLEST INPUT that still triggers the failure — the corpus's claim is that this *"usually is the diagnosis"*, not that it IS the root cause; the root cause remains §11.4.102's phase. Recommended mechanism gate `CM-DELTA-DEBUG-MINIMAL-CASE` (an investigation of a non-monotone or non-commit-discriminated failure records the narrowed axis and its minimal failing case, and that case is the RED test's input; a fix designed against an un-narrowed failing input → FAIL) + paired §1.1 mutation (strip the minimal-case record so the fix is designed against the full unreduced input → the gate MUST FAIL; golden-FALSE per §11.4.201(1): a genuine monotone commit-range regression localised by `git bisect` alone MUST NOT fire it). Gate-code = separate work item, NOT claimed shipped (§11.4.6 / §11.4.227).

Canonical authority: constitution submodule `Constitution.md` §11.4.242. Non-compliance is a release blocker. No escape hatch — no `--blame-log-suffices`, `--skip-bisection`, `--flaky-observable-bisect-anyway`, `--run-script-exit-code-collapse-OK`, `--regression-guess-without-localisation` flag.

2026 08 15

§ . . — Characterization / golden-master safety net for legacy code before behaviour-changing modification (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.243. Before ANY behaviour-changing modification to legacy code lacking characterization tests (code inherited without written behavioural contract, code whose original author has moved on, code whose documented behaviour has diverged from actual behaviour, code that "works but nobody knows why"), the modifier MUST FIRST author CHARACTERIZATION TESTS asserting CURRENT observable behaviour as GROUND TRUTH (whether "correct" or not) and land them as a green baseline. Only AFTER the safety net exists may the modification proceed. Pattern originates in Feathers's WORKING EFFECTIVELY WITH LEGACY CODE + generalises GOLDEN-MASTER (record outputs across representative input space; assert future runs match). LEGACY SCOPE (§11.4.6-honest): (i) lacks tests asserting observable behaviour; OR (ii) written spec diverges from measured actual behaviour; OR (iii) predates the constitution's TDD discipline (§11.4.224); OR (iv) modifier cannot describe its exact behaviour from reading source in time budget. AUTHORING LOOP: (1) identify observable surface → (2) generate representative input space → (3) run + record actual outputs as ground-truth baseline → (4) assert recorded values in tests → (5) commit characterization tests as SEPARATE commit BEFORE the behaviour-changing modification. DISTINCTION FROM CORRECTNESS: characterization tests capture WHAT IS not WHAT SHOULD BE — DESCRIPTIVE not prescriptive. If current legacy behaviour is subtly wrong, the characterization test CAPTURES that wrongness; the modifier's job is to notice, ASK the operator per §11.4.66 whether to preserve or correct that behaviour, never silently. Composes §11.4.115 (RED-first — characterization is legacy counterpart) / §11.4.124 (investigate-before-remove — parallel discipline) / §11.4.118 (sampled input space = honest gaps, not completeness) / §11.4.201 / §11.4.224 (test-first

backfilled for legacy). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-243-PROPAGATION` (literal `11.4.243`) + recommended gate `CM-CHARACTERIZATION-BEFORE-LEGACY-MODIFICATION` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.243. Non-compliance is a release blocker. No escape hatch — no `--modify-untested-legacy-directly`, `--skip-characterization`, `--claim-tests-not-needed-because-obvious`, `--decorative-assertion-suffices`, `--refactor-without-safety-net` flag.

2026 08 15

§ . . — Cross-boundary contract tests + can-i-deploy gate (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.244. Changes modifying a boundary between independently-deployable components (service↔consumer, library↔dependents, DB schema↔readers/writers, message topic↔subscribers) MUST be covered by CONSUMER-DRIVEN CONTRACT TESTS on both sides AND a `can-i-deploy` gate MUST verify proposed provider version compatibility against every currently-deployed consumer version BEFORE the change ships. §11.4.244 MERGES contract-tests + can-i-deploy (they are useless in isolation — contract tests without the gate produce compatibility knowledge nobody consults; a can-i-deploy gate without contract tests has nothing to check). CDC pattern: consumers publish assumptions they make about the provider (Pact / Spring Cloud Contract / gRPC schema-evolution rules); providers verify each consumer's contract against actual implementation; provider change breaking a live consumer's contract fails provider's build BEFORE the change ships — "a precise red at the source." CAN-I-DEPLOY: at every deploy attempt, gate queries the contract broker for the compatibility matrix; single red cell BLOCKS the deploy with machine-readable verdict naming the incompatible pair, NEVER warning-with-continue. MISSING contract = first-class refusal reason per §11.4.201 (a false-positive REFUSAL is a FAIL-bluff, false-negative allowance is a PASS-bluff). SemVer becomes MEASURABLE (PATCH bump breaking contract = mis-labelled version). Backward-compatibility window declared per §11.4.35. Composes §11.4.5 / §11.4.6 / §11.4.35 / §11.4.66 / §11.4.108 (per-side four-layer) / §11.4.135 (contract tests as permanent regression guards) / §11.4.185 (composed-system manual QA

still needed) / §11.4.201 (gate honesty) / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-244-PROPAGATION` (literal `11.4.244`) + recommended gates `CM-CONTRACT-TESTS-BOTH-SIDES` + `CM-CAN-I-DEPLOY-GATE-WIRED` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.244. Non-compliance is a release blocker. No escape hatch — no `--skip-contract-tests`, `--deploy-without-compatibility-check`, `--assume-compatible-when-broker-silent`, `--patch-may-break-contract`, `--contract-tests-optional-for-internal-boundaries` flag.

2026 08 15

§ . . — Oracle-problem-first test authoring: identify the oracle BEFORE authoring the test (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.245. Every test MUST FIRST identify its ORACLE — the mechanism deciding PASS vs FAIL for the input it drives — and the oracle MUST be DIFFERENT from the code under test AND from the test's own assertion mechanism. Authoring without explicit oracle identification produces the "test agrees with the code" anti-pattern (test asserts what the code happens to produce, so it passes as long as the code produces something, never catches wrong-output — the most common form of §11.4/§11.4.1 bluff at the test-authoring layer). The oracle problem is the OLDEST unsolved problem in software testing; §11.4.245 does not solve it, it mandates HONESTLY IDENTIFYING which oracle strategy each test uses so the reviewer per §11.4.194 can judge whether the identified oracle is genuinely independent. ORACLE STRATEGY CLOSED SET (each test's oracle NAMED explicitly in source): (1) SPECIFIED (spec/RFC/standard names expected output); (2) DERIVED (independent reference implementation); (3) METAMORPHIC (metamorphic relations between related inputs — §11.4.107(8) generalised); (4) GOLDEN-MASTER / CHARACTERIZATION (past captured behavior — §11.4.243); (5) INVARIANT (universal property must hold — sort output sorted + permutation of input); (6) STATISTICAL (distribution properties for stochastic outputs, never specific value); (7) HUMAN (§11.4.185 manual QA). NO other classes. STRUCTURAL INDEPENDENCE: oracle's SOURCE of expected value does not depend on code

being tested. §11.4.107(10) SELF-VALIDATION generalised: paired §1.1 mutation must produce a WRONG output; oracle catches it. Composes §11.4.5 / §11.4.6 / §11.4.107(10) / §11.4.115 / §11.4.194 / §11.4.201 / §11.4.224 / §11.4.243 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-245-PROPAGATION` (literal `11.4.245`) + recommended gate `CM-ORACLE-STRATEGY-NAMED-AND-INDEPENDENT` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.245. Non-compliance is a release blocker. No escape hatch — no `--test-agrees-with-code`, `--skip-oracle-identification`, `--oracle-is-the-code`, `--assertion-is-oracle`, `--implicit-oracle-OK` flag.

2026 08 15

§ . . — Reproducible + hermetic builds + supply-chain integrity at SLSA Build Level (operator decision, - -)

Full anchor per Phase 3 landing brief §11.4.246. Every consuming project's build pipeline MUST produce REPRODUCIBLE, HERMETIC artifacts and MUST meet SLSA Build Level 2 as the FLEET-WIDE MINIMUM (operator decision, 2026-08-15 — Level 3+ tracked as backlog per §11.4.197). REPRODUCIBLE = same source at same time on any conformant host produces byte-identical artifacts (or diff-set fully accounted for by declared non-determinisms pinned to canonical values). HERMETIC = no undeclared inputs (every dep version-pinned + lockfile-hashed, every tool version declared, no unpinned public-network reach at build, no host-env leakage). SLSA-L2 = signed provenance describing source URI + commit hash + builder identity + invocation + artifact hash, on hosted tamper-resistant build platform (boba already satisfies ancillary: SSH-only §2.1, rootless podman §11.4.161, fixed-tag pins §11.4.30/§11.4.77). Every consumer MAINTAINS `docs/security/SLSA_LEVEL.md` (or per §11.4.35 equivalent) recording CURRENT L level with cited evidence + TARGET L + tracked §11.4.197 upgrade items — CURRENT below L2 without tracked item is violation. SUPPLY-CHAIN EXTENSION: deps either vendored (checked in hash-verified), from provenance-attested source (npm/PyPI/crates.io as they add SLSA), OR mirrored through hash-pinning local registry. Threat model NARROW: covers source→artifact chain, does NOT cover malicious commit landing in source (§11.4.142 review) nor

compromised dep source nor bug-free artifact (§11.4.108 four-layer + §11.4.185 manual QA). Composes §2.1 / §11.4.6 / §11.4.30 / §11.4.35 / §11.4.108 / §11.4.113 / §11.4.135 / §11.4.142 / §11.4.161 / §11.4.184 / §11.4.185 / §11.4.197 / §11.4.209 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-246-PROPAGATION` (literal `11.4.246`) + recommended gates `CM-BUILD-REPRODUCIBILITY` + `CM-BUILD-HERMETICITY` + `CM-SLSA-L2-PROVENANCE-VERIFIED` + `CM-SUPPLY-CHAIN-DEPS-PROVENANCE-ATTESTED` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.246. Non-compliance is a release blocker. No escape hatch — no `--skip-reproducibility`, `--allow-network-during-build`, `--no-provenance`, `--slsa-optional`, `--below-l2-OK`, `--unpinned-dependency-OK`, `--host-env-may-leak-into-build` flag.

2026 08 15

§ . . — Composite-output layer-move must move every layer (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.247. Changes MOVING a system-boundary layer (service migrates hosts, endpoint URL changes, API version bumps, DB schema shifts, message topic renames, ingress path re-routes, storage bucket moves) MUST move EVERY LAYER of the composite output atomically or via explicit compatibility window — NEVER piecemeal. LAYER TAXONOMY per §11.4.35 baseline: (i) server (handler + route + OpenAPI/proto/gRPC descriptor); (ii) client (call-sites + SDK stubs + integration tests); (iii) infrastructure (LB + ingress + API gateway + CDN + service-mesh + DNS); (iv) data stores (cached keys + DB URL rows + MQ subscriptions); (v) observability (metric labels + span names + log fields + dashboards + alerts + SLOs); (vi) documentation (user + dev + runbook + README + cookbook); (vii) tests (integration + e2e + contract per §11.4.244 + load + smoke). Consumer provides `scripts/audit/layer_move_completeness.sh` reporting residual references delta; change proposing move MUST show ZERO residuals OR explicit compatibility-window plan per residual with tracked §11.4.197 items. COMPATIBILITY-WINDOW: old value keeps working alongside new until stated date/version/event, deprecation warnings named per caller, tracked deadline, clean removal at expiry — no "clean up later." [MATERIAL-THIN: Module

32 ("Secondary-Display & External-Output Media Pipelines", whose "Surface lifecycle: the heart of the reopening problem" section covers this domain) supplies the composite-output pattern (multiple co-varying layers, atomic move discipline, surface-swap-not-recreate); the specific web/service layer inventory is generalised from the pattern's shape per §11.4.17 with consumer §11.4.35 DATA supplying the concrete inventory. **CITATION AUDIT 2026-08-20 (§11.4.6):** the three named phrases — "composite-output"/"composite output", "surface-swap-not-recreate"/"surface-swap", and "atomic move discipline" — have ZERO occurrences in ALL NINE tracked corpus modules (instrument control-needled per §11.4.201(7)(b) before the absence was reported). They are this anchor's OWN PARAPHRASE of the module-32 domain, NOT corpus quotations, and MUST NOT be read as quoted source text. The nearest phrase-level in-corpus material on surface swapping is `module_35:95` / `module_35:102` (`setOutputSurface` / `BAD_INDEX (6)`) — module 35, not module 32. The module-32 DOMAIN attribution is plausible and unrefuted; the PHRASE-level attribution is UNVERIFIED.] Composes §11.4.6 / §11.4.35 / §11.4.111 / §11.4.118 / §11.4.124 / §11.4.135 / §11.4.191 / §11.4.197 / §11.4.238 (residual = coverage escape) / §11.4.244 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-247-PROPAGATION` (literal `11.4.247`) + recommended gate `CM-LAYER-MOVE-COMPLETENESS-CHECK` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.247. Non-compliance is a release blocker. No escape hatch — no `--partial-layer-move-OK` , `--skip-completeness-check` , `--residuals-cleaned-up-later` , `--observability-layer-optional` , `--doc-move-can-lag` flag.

2026 08 15

§ . . — Flaky-test quarantine + protected-regression-spec gate via `[PROTECTED-SPEC: ATM-NNN]` tag + CODEOWNERS required reviewer (operator decision, - -)

Full anchor per Phase 3 landing brief §11.4.248. Flaky tests are the SILENT ANTI-BLUFF DECAY vector (every flake conditions team to ignore red until real regression is dismissed as "probably flaky, re-run"). Two disciplines bound: (A) QUARANTINE — mechanical detection (test-history aggregator flags two-consecutive-run-different-verdict OR rerun-after-red pattern) moves flaky test to

`tests/quarantine/` (per §11.4.35), opens tracked §11.4.197 stabilisation deadline, blocking suite runs WITHOUT quarantined test so green means green; deadline past = §11.4.66 extension or violation; (B) PROTECTED-REGRESSION-SPEC — regression tests codifying known-broken defect fixes (per §11.4.135 permanent-guard suite) MAY be tagged `# [PROTECTED-SPEC: ATM-NNN]` (target-language comment equivalent) referencing the workable-item ATM-NNN (§11.4.54) they guard, marking test as load-bearing regression guard whose modification requires elevated review. Tag applied deliberately per test (not blanket — that voids the discipline). CODEOWNERS-REQUIRED-REVIEWER GATE: consuming project's CODEOWNERS declares modifications to `[PROTECTED-SPEC]`-tagged tests OR `regression/**/*` path scope require designated required-reviewer approval BEFORE merge; composes with §2.1 SSH + §11.4.113 no-force-push (gate cannot be silently bypassed); audit trail in PR review thread + commit history. Gate CATCHES change so reviewer distinguishes STRENGTHEN (approve) vs FIX-DEFECT-IN-TEST (approve after verifying ATM-NNN's defect not silently re-introduced) vs WEAKEN/REMOVE (refuse or §11.4.66 escalate) — makes modification VISIBLE not preventable. Composes §2.1 / §11.4.6 / §11.4.35 / §11.4.50 (ideal empty quarantine) / §11.4.54 / §11.4.66 / §11.4.113 / §11.4.115 / §11.4.135 / §11.4.197 / §11.4.226 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-248-PROPAGATION` (literal `11.4.248`) + recommended gates `CM-FLAKY-TEST-QUARANTINE-WIRED` + `CM-PROTECTED-REGRESSION-SPEC-GATE` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.248. Non-compliance is a release blocker. No escape hatch — no `--rerun-until-green`, `--quarantine-permanent`, `--protected-spec-review-optional`, `--codeowners-skip`, `--blanket-protect-everything`, `--tag-without-atm-id` flag. [4] [249]

2026 08 15

§ . . — Producer ≠ oracle ≠ gate ≠ verifier + flight recorder (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.249. Every load-bearing quality mechanism has FOUR distinct roles that MUST be structurally separate — (1) PRODUCER (code computing/emitting/building output); (2) ORACLE (mechanism

deciding correctness per §11.4.245); (3) GATE (seam consuming oracle verdict to allow/block/proceed); (4) VERIFIER (mechanism auditing gate's decision — paired §1.1 mutation proving gate can refuse a bad output). Collapsing any two produces specific bluff class: producer=oracle (§11.4.245 oracle-agrees-with-code); oracle=gate (refuses to say "cannot decide" defaults to allow, §11.4.201(6) FALSE-NULL); gate=verifier (self-auditing gate — §11.4.142 self-review anti-pattern applied). FLIGHT RECORDER: mechanism CAPTURING producer inputs + oracle verdict + gate decision + verifier audit as durable replayable artifact (structured JSONL event stream per §11.4.116, content-addressed per §11.4.207) so post-hoc "why did this go through" has evidence not shrug. Recorder is NOT the gate — a gate reading its own recorder to decide collapses gate + verifier + recorder. DETECTABLE COLLAPSES reviewer per §11.4.194 scans for: (1) producer=oracle (test's assertion calls code being tested), (2) oracle=gate (no "cannot decide" branch, defaults PASS), (3) gate=verifier (meta-test IS the gate — mutation making gate always-pass ALSO makes meta-test always-pass; meta-test MUST live OUTSIDE gate), (4) recorder=decision-substrate (gate uses recorder contents as decision input — precedent-based gating not authorised). [MATERIAL-THIN: four-role taxonomy generalised from producer/verifier separation + oracle-problem discussion + gate mechanics; FLIGHT RECORDER derived from clean-checkout re-run + attached receipts pattern per §11.4.17.] Composes §11.4.5 / §11.4.6 / §11.4.107(10) / §11.4.116 / §11.4.142 / §11.4.194 / §11.4.201 / §11.4.207 / §11.4.240 (producer↔verifier separation principle — mutual composes per §11.4.209 reviewer MINOR-1) / §11.4.245 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-249-PROPAGATION` (literal `11.4.249`) + recommended gates `CM-ROLE-SEPARATION-DECLARED` + `CM-FLIGHT-RECORDER-CAPTURED` + paired §1.1 mutations.

Canonical authority: constitution submodule `Constitution.md` §11.4.249. Non-compliance is a release blocker. No escape hatch — no `--collapse-producer-oracle`, `--gate-self-audits`, `--skip-flight-recorder`, `--oracle-defaults-pass`, `--decision-without-recorded-rationale` flag.

---

§ . . — Heuristic-tower signals a primitive defect: when N heuristic layers stack to compensate, the primitive is broken (research-derived, - -)

Full anchor per Phase 3 landing brief §11.4.250. Codebase accumulating TOWER of heuristics compensating for same underlying issue (retry-with-backoff + connection-pool-warmup + DNS-cache-flush + health-check-with-startup-grace — all guarding one flaky DB connection) IS the diagnostic: PRIMITIVE is defective, every layer papers over a manifestation without fixing cause. Extraction line 6486 [offset un-resolvable per the §11.4.239 CITATION NOTE; **MODULE ADDED 2026-08-20**]: heuristic-compensating-for-broken-primitive taxonomy row + ATMOSphere `BAD_INDEX (6)` compensator-tree case study — resolves to **module 35** taxonomy row 4 "Heuristic compensating for a broken primitive", verified locators `module_35:99` (row header), `module_35:102` (the ATMOSphere `setOutputSurface()` / `BAD_INDEX (6)` compensator case), `module_35:104` (the attack-the-primitive remedy). **OBSERVED CORRELATION (§11.4.6 — an observation, NOT a proven mapping):** all four offsets cited across §11.4.250 / §11.4.251 / §11.4.254 land on module 35 taxonomy rows under a constant offset of 6482 (6486→row 4, 6488→row 6, 6489→row 7, 6490→row 8; 4 of 4 fit), consistent with module 35's taxonomy beginning near offset 6482 in the consolidated extraction artifact; that artifact is not tracked in this repository, so the mapping cannot be confirmed here and is not asserted as fact. Recognition signals: (i) each layer justification "we added because sometimes X happens" — reactive; (ii) removing any layer INCREASES failures BUT keeping all shows occasional residual; (iii) no single layer owns responsibility; (iv) layers added at different times by different authors per specific incidents; (v) monotone layer-count growth over time — debugging session for residual adds layer N+1 rather than fixing 1..N. DISTINCT from defense-in-depth: defense-in-depth is deliberate + each layer targets DIFFERENT failure class + designed together not accreted + removing layer produces SPECIFIC different failure not "same failure more often." SYSTEMATIC-DEBUGGING RESPONSE on tower depth ≥ 2: (1) STOP adding layers; (2) INVOKE §11.4.102 systematic-debugging on PRIMITIVE; (3) proceed root-cause / pattern / hypothesis / implementation phases; (4) fix primitive; (5) REMOVE layers one at a time verifying with §11.4.5 evidence; (6) commit each removal per §11.4.124 with git-history evidence citing primitive fix. Composes §11.4.5 / §11.4.6 / §11.4.102 / §11.4.124 / §11.4.194 / §11.4.201 (outer layer masks

§11.4.201(6) false-null) / §11.4.226 / §1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-250-PROPAGATION` (literal `11.4.250`) + recommended gate `CM-HEURISTIC-TOWER-TRIGGERS-PRIMITIVE-DEBUG` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.250. Non-compliance is a release blocker. No escape hatch — no `--add-another-heuristic-layer`, `--skip-primitive-debug`, `--tower-is-defense-in-depth-without-justification`, `--reactive-mitigation-OK`, `--layer-per-incident` flag.
